

From Reinforcement Learning to Bayesian Optimization

Puyao Ge, Hengrui Luo

Not peer reviewed.
September 1, 2026 Ver 0.5

Contents

Preface	4
1 Discrete-Time Markov Chains	6
1.1 Stochastic Processes and the Markov Property	
1.2 Transition Probabilities and Matrices	
1.3 Constructing DTMCs	
1.4 Transient Probabilities and Occupancy	
1.5 Stationary and Limiting Behavior	
1.6 Rewards and Long-Run Averages	
2 Exponential Clocks and Continuous Time Chains	13
2.1 Exponential Clocks	
2.2 Competing Exponential Clocks	
2.3 Poisson Arrivals	
2.4 Continuous Time Markov Chains	
2.5 Sojourn Times	
2.6 Generator Matrices	
3 Birth and Death Processes and Queues	19
3.1 Birth and Death Structure	
3.2 Queueing Models	
3.3 $M/M/1$ Queue	
3.4 Transient Probabilities	
3.5 Stationary Behavior and Applications	
3.6 Rewards, Costs, and Net Profit	
4 Finite Horizon MDPs and Backward Induction	26
4.1 States, Actions, and Rewards	
4.2 Histories and Policies	
4.3 Backward Recursion	
4.4 Bellman Equation	
4.5 Recovering an Optimal Policy	
4.6 Stopping as a Finite Horizon MDP	
5 Finite Horizon Resource and Reliability Models	34
5.1 State Balance and One Period Cost	
5.2 Inventory Recursion and Order Up To Decisions	
5.3 Convexity and Base Stock Structure	
5.4 Reliability States and Replacement Actions	
5.5 Computing the Backward Recursion	
5.6 Monotonicity and Threshold Policies	
6 Finite Horizon Optimal Stopping	41
6.1 Stopping Decisions and Sufficient States	
6.2 Bellman Recursion for Sequential Selection	
6.3 Threshold Structure and the Large Sample Rule	
6.4 Parking as a Cost Minimization Problem	
6.5 Comparing the Two Stopping Rules	

7	Finite Horizon Examples	46
7.1	Service Control and State Choice	
7.2	Arrivals and State Transitions	
7.3	Clinic Bellman Recursion	
7.4	Sports Control as a Finite Horizon Decision	
7.5	Transition Probabilities and Bellman Equation	
7.6	Data Calibration and Policy Interpretation	
8	Discounted MDPs and Bellman Operators	52
8.1	Discounted Return and Policy Value	
8.2	Optimal Value Function	
8.3	Bellman Equation and Policy Recovery	
8.4	Bellman Operator and Contraction	
8.5	State Augmentation in a Financial Model	
9	Computing Discounted Policies	57
9.1	Value Iteration	
9.2	Policy Iteration	
9.3	Primal Linear Program	
9.4	Occupation Measures and the Dual Program	
10	Continuous Time Control and Long Run Criteria	61
10.1	Decisions at Continuous Time Events	
10.2	Discounting Between Events	
10.3	The Average Reward Criterion	
10.4	From Discounted to Average Reward	
11	Average Reward Bellman Equations and Algorithms	65
11.1	Average Reward and Relative Value	
11.2	The Average Reward Bellman Equation	
11.3	The Discounted Limit	
11.4	Solving a Finite Model	
11.5	Relative Value Iteration	
11.6	Linear Programs and Occupation Measures	
12	Learning Value Functions from Experience	69
12.1	From a Known Model to Sampled Experience	
12.2	Model Estimation and Direct Value Learning	
12.3	Monte Carlo Policy Evaluation	
12.4	Temporal Difference Learning	
12.5	Q Learning and Exploration	
13	Bandits and Directed Exploration	72
13.1	The Bandit Model and Regret	
13.2	Empirical Means and Random Exploration	
13.3	Upper Confidence Bounds	
13.4	Thompson Sampling	
13.5	What Carries Forward to Bayesian Optimization	
14	Function Approximation and Policy Gradients	75

14.1	Why Tables Stop Scaling	
14.2	Linear Value Approximation	
14.3	Nonlinear Q Approximation	
14.4	Differentiable Policies	
14.5	REINFORCE and Actor and Critic Methods	
15	Experimental Design, Contraction, and Regret	78
15.1	Prediction and Decision Goals	
15.2	Linear Designs and Information Matrices	
15.3	Adaptive Design as an Information State	
15.4	Regret Measures Decision Loss	
15.5	Uniform Contraction Gives Simple Regret	
15.6	Sequential Widths Give Cumulative Regret	
16	Gaussian Process Surrogates	82
16.1	A Prior Over Functions	
16.2	Kernels and Shared Information	
16.3	Posterior Conditioning Formulas	
16.4	Posterior Uncertainty and Updates	
16.5	Design by Variance Reduction	
17	Bayesian Optimization: Information States and Acquisition Rules	86
17.1	The Bayesian Optimization Loop and Information State	
17.2	Upper Confidence Bound	
17.3	Expected Improvement	
17.4	Probability of Improvement and Posterior Sampling	
17.5	Optimizing the Acquisition and Stopping	
18	A Common View of MDPs and Bayesian Surrogates	90
18.1	A Common Decision Cycle and Two Kinds of State	
18.2	Bellman Values and Acquisition Scores	
18.3	Exploration, Uncertainty, and Regret	
18.4	Choosing the Appropriate Model	
19	Treatment and Inventory as Decision Models	93
19.1	Begin With the Decision Question	
19.2	Treatment Over Time as an MDP	
19.3	Inventory and Pricing as Sequential or Design Problems	
19.4	Assumptions That Change the Answer	
20	Simulation Policy Tuning and Regret	96
20.1	Inner and Outer Decisions with Policy Parameters	
20.2	Noisy Evaluation and Replication	
20.3	A Bayesian Optimization Workflow	
20.4	Regret, Validation, and the Contraction Connection	
	References	98

Preface

These notes combine material from CMOR 544 and STAT 620 taught at Rice University during 2025AU–2026SP around one theme: sequential decision-making under uncertainty.

CMOR 544 targets undergraduates and graduate students, providing a foundation in sequential decision-making with state dynamics. In the lecture notes, the main objects are stochastic processes, Markov chains, and Markov decision processes. A decision maker observes a state, chooses an action, receives a reward, and the system transitions to a future state according to Markovian dynamics. The course covers finite-horizon and infinite-horizon MDPs, Bellman equations, backward recursion, value iteration, policy iteration, linear programming formulations, average-reward problems, and reinforcement learning methods such as Monte Carlo learning, temporal-difference learning, Q-learning, exploration-exploitation, bandits, UCB, Thompson sampling, function approximation, policy gradients, REINFORCE, and actor-critic methods.

STAT 620 targets at graduate students, which extends this decision-theoretic viewpoint toward Bayesian optimization, experimental design, and statistical reinforcement learning. In the lecture notes, the main objects are expensive black-box functions, design points, surrogate models, Gaussian processes, acquisition functions, bandit algorithms, and regret bounds. Instead of focusing primarily on physical state transitions, the course asks how to choose experiments or actions sequentially in order to learn an unknown reward or objective function efficiently. It develops optimal experimental design, linear and kernel regression designs, Gaussian process posterior inference, Bayesian optimization, Thompson sampling, UCB-type methods, Monte Carlo tree search, high-dimensional and multi-objective Bayesian optimization, concentration inequalities, stochastic and adversarial bandits, contextual and linear bandits, and Bayesian bandits.

These notes assume linear algebra, probability, and multivariable calculus.

The recurring problem is to choose an action, observe what it reveals, update what is known, and choose again. The main model in CMOR 544 is the Markov decision process, the main equation is the Bellman equation, and the main computational methods are dynamic programming and reinforcement learning. The STAT 620 material is used as a bridge in two related ways: from one-state MDPs to bandits, and from classical experimental design to Bayesian optimization as adaptive design. The two anchor models are:

1. MDP / reinforcement learning model
2. one-state MDP / bandit / Bayesian optimization model

An MDP allows actions to change both rewards and future states. A bandit removes state dynamics. Bayesian optimization keeps the same sequential action-selection problem, but the action is now a design point and the unknown reward function is modeled with a statistical surrogate, often a Gaussian process. The path is Markov chains $\rightarrow$ Markov decision processes (MDPs) $\rightarrow$ reinforcement learning and bandits $\rightarrow$ experimental design $\rightarrow$ Gaussian processes (GPs) $\rightarrow$ Bayesian optimization (BO). MDPs supply the language of states, actions, policies, and future consequences. Bandits isolate learning while acting. Experimental design asks which observation is useful. A GP gives a probability model for an unknown response surface, and BO uses that model to select expensive experiments. Standard references are Puterman [1], Bertsekas [2], Sutton and Barto [3], Santner et al. [4], Rasmussen and Williams [5], and Frazier [6], Shahriari et al. [7].

AI use declaration: When preparing course materials and teaching, authors did not use AI; when preparing this manuscript, the authors use GPT-4o/GPT5 for latex formatting, grammar checks and language editing. The authors are responsible for the materials and comments/erratas are welcome.

Common notation

The notation below gives our default conventions; when a symbol is defined differently within a section, the local definition takes precedence.

Symbol	Meaning
t, n	time or sequential round; n also denotes a fixed sample size when stated
$X_n, X(t)$	state of an uncontrolled discrete or continuous time process
S_t, A_t, G_{t+1}	controlled state, action, and observed reward after the action
$g(s, a), g_{\text{term}}(s)$	expected one period reward and terminal reward
$V_m(s), \pi$	value with m decisions remaining and a policy
$\rho, h(s)$	long run average reward and relative value
a_t, μ_a	selected bandit arm and its mean reward
$x_t, y_t = f_0(x_t) + \xi_t$	design point and noisy observed response
$\mathcal{D}_t = \{(x_i, y_i) : 1 \leq i \leq t\}$	data set, used as the information state
$m_t(x), s_t(x)$	posterior mean and posterior standard deviation
$x^*, \hat{x}_T$	true optimizer and final recommended design
r_t, R_T, r_T^s	instantaneous, cumulative, and simple regret; these symbols are not used for MDP rewards
ϵ_n	contraction or estimation error radius, not observation noise

Uppercase letters denote random quantities when the distinction matters. We use X for an uncontrolled chain and S for a controlled state; application specific examples may name the physical quantity directly. A policy is denoted by π , while f_0 denotes the unknown response surface in design and BO. Transition matrix entries are $p_{ij}(a)$ and the corresponding matrix is $P(a)$. Vectors are columns; $\|v\|_2$ is the Euclidean norm; and $K^{-1}v$ means solving $Ku = v$, not explicitly forming an inverse. The expression $\arg \max_{x \in \mathcal{X}} f_0(x)$ denotes an optimizing input, whereas $\max_{x \in \mathcal{X}} f_0(x)$ denotes the attained value. We maximize f_0 ; a minimization problem is obtained by replacing f_0 with $-f_0$.

Lecture 1 Discrete-Time Markov Chains

We begin with the general notion of a stochastic process.

A stochastic process is a collection of random variables

$$\{X(t), t \in T\},$$

indexed by a parameter t , where T is called the *index set* or *parameter set*. For each $t \in T$, the random variable $X(t)$ takes values in a set S , called the *state space*.

1.1 Stochastic Processes and the Markov Property

Definition

A stochastic process $\{X_n, n \geq 0\}$ is called a *discrete-time stochastic process* if the system is observed at discrete time points

$$n = 0, 1, 2, \dots,$$

and X_n denotes the state of the system at time n .

An example of a discrete-time stochastic process:

- the Dow Jones index at the end of each week.

Let $\{X_n, n \geq 0\}$ be a discrete-time stochastic process. Suppose that the system is observed at some fixed time m . Then:

- X_m denotes the present state of the system;
- $\{X_0, X_1, \dots, X_{m-1}\}$ denotes the history of the system;
- $\{X_{m+1}, X_{m+2}, \dots\}$ denotes the future evolution of the system.

Definition

A discrete-time stochastic process $\{X_n, n \geq 0\}$ is said to satisfy the *Markov property* if, conditional on the present state of the system, the future evolution of the system is independent of its history.

If $\{X_n, n \geq 0\}$ satisfies the Markov property and has a finite or countable state space S , then it is called a *discrete-time Markov chain*, or *DTMC*.

The Markov property can be written more formally as follows. For all $n \geq 0$ and all states $j, i, i_{n-1}, \dots, i_0 \in S$,

$$\mathbb{P}(X_{n+1} = j \mid X_n = i, X_{n-1} = i_{n-1}, \dots, X_0 = i_0) = \mathbb{P}(X_{n+1} = j \mid X_n = i),$$

whenever the conditional probabilities are well defined.

1.2 Transition Probabilities and Matrices

Define the one-step transition probability from state i to state j at time n by

$$p_{ij}(n) = \mathbb{P}(X_{n+1} = j \mid X_n = i).$$

Definition

A DTMC $\{X_n, n \geq 0\}$ is called *time-homogeneous* if the one-step transition probabilities do not depend on time n . That is,

$$\mathbb{P}(X_{n+1} = j \mid X_n = i)$$

is independent of n , so that

$$p_{ij}(n) = p_{ij}.$$

For a time-homogeneous DTMC, the transition probabilities satisfy $p_{ij} \geq 0$ and

$$\sum_{j \in S} p_{ij} = 1, \quad \forall i \in S.$$

For a time-homogeneous DTMC, let P denote the transition probability matrix:

$$P = (p_{ij})_{i,j \in S}.$$

Each row of P sums to one.

1.3 Constructing DTMCs

Example 1.1: Transition Diagram

Transition diagrams provide graphical representations of time-homogeneous DTMCs. Consider a DTMC with state space

$$S = \{1, 2, 3\}$$

and transition probability matrix

$$P = \begin{pmatrix} 0.2 & 0.8 & 0 \\ 0 & 0.3 & 0.7 \\ 0.1 & 0.4 & 0.5 \end{pmatrix}.$$

The entry p_{ij} gives the probability of moving from state i to state j in one step. Therefore, each positive entry of P corresponds to a directed edge from state i to state j , labeled by p_{ij} .

Example 1.2: Success Runs

Consider a game in which a coin is tossed independently over and over again. Each time the coin turns up heads, which occurs with probability p , the player wins one dollar. Whenever the coin turns up tails, the player loses all earnings accumulated up to that point.

Let $X_0 = 0$, and let X_n denote the player's earnings after the n th toss for $n \geq 1$. Then $\{X_n, n \geq 0\}$ is a DTMC with state space

$$S = \{0, 1, 2, \dots\}.$$

Given the current state $X_n = i$, the next state is

$$X_{n+1} = \begin{cases} i + 1, & \text{with probability } p, \\ 0, & \text{with probability } 1 - p. \end{cases}$$

Therefore, the one-step transition probabilities are

$$p_{i,i+1} = p, \quad p_{i,0} = 1 - p, \quad i = 0, 1, 2, \dots,$$

and all other transition probabilities are zero.

1.4 Transient Probabilities and Occupancy

We now study the transient behavior of a time-homogeneous DTMC. The word *transient* here refers to the behavior of the chain over a finite time horizon.

Let

$$a_i = \mathbb{P}(X_0 = i), \quad i \in S.$$

The vector

$$\mathbf{a} = (a_i)_{i \in S}$$

is called the *initial distribution* of the DTMC.

For $n \geq 0$, define

$$p_{ij}^{(n)} = \mathbb{P}(X_n = j \mid X_0 = i),$$

which is called the n -step transition probability from state i to state j . By convention,

$$p_{ij}^{(0)} = \begin{cases} 1, & i = j, \\ 0, & i \neq j. \end{cases}$$

Let $V_j^{(n)}$ denote the number of visits to state j by the DTMC over the time set

$$\{0, 1, 2, \dots, n\},$$

including a possible visit at time 0. Equivalently,

$$V_j^{(n)} = \sum_{k=0}^n \mathbf{1}\{X_k = j\}.$$

Define

$$M_{ij}^{(n)} = \mathbb{E} \left[V_j^{(n)} \mid X_0 = i \right],$$

which is the expected occupancy time of state j up to time n , starting from state i .

Theorem

For any states $i, j \in S$ and any $n \geq 0$,

$$M_{ij}^{(n)} = \sum_{k=0}^n p_{ij}^{(k)}.$$

Proof. Since

$$V_j^{(n)} = \sum_{k=0}^n \mathbf{1}\{X_k = j\},$$

we have

$$M_{ij}^{(n)} = \mathbb{E} \left[\sum_{k=0}^n \mathbf{1}\{X_k = j\} \mid X_0 = i \right].$$

By linearity of expectation,

$$M_{ij}^{(n)} = \sum_{k=0}^n \mathbb{E} [\mathbf{1}\{X_k = j\} \mid X_0 = i].$$

For each k ,

$$\mathbb{E}[\mathbf{1}\{X_k = j\} \mid X_0 = i] = \mathbb{P}(X_k = j \mid X_0 = i) = p_{ij}^{(k)}.$$

Therefore,

$$M_{ij}^{(n)} = \sum_{k=0}^n p_{ij}^{(k)}.$$

□

1.5 Stationary and Limiting Behavior

Next, we consider long-run behavior. A probability distribution

$$\boldsymbol{\pi} = (\pi_j)_{j \in S}$$

on S is called a *stationary distribution* of a time homogeneous DTMC with transition matrix P if

$$\boldsymbol{\pi}P = \boldsymbol{\pi}.$$

Equivalently,

$$\pi_j = \sum_{i \in S} \pi_i p_{ij}, \quad j \in S.$$

Suppose that the initial distribution is the stationary distribution, that is,

$$a_j = \mathbb{P}(X_0 = j) = \pi_j, \quad j \in S.$$

Then, by induction,

$$\mathbb{P}(X_n = j) = \pi_j, \quad j \in S,$$

for every $n \geq 1$. Thus, if the chain starts in stationarity, its marginal distribution remains unchanged over time.

The expected average occupancy time of state j , starting from state i , is

$$\frac{1}{n+1} M_{ij}^{(n)} = \frac{1}{n+1} \sum_{k=0}^n p_{ij}^{(k)}.$$

For a finite irreducible (every state can eventually reach every other state) DTMC, the limiting occupancy distribution is given by the stationary distribution. That is,

$$\lim_{n \rightarrow \infty} \frac{1}{n+1} M_{ij}^{(n)} = \lim_{n \rightarrow \infty} \frac{1}{n+1} \sum_{k=0}^n p_{ij}^{(k)} = \pi_j.$$

Thus, the long run fraction of time spent in state j is π_j .

Example 1.3: Two State DTMC

Consider a two state DTMC with state space

$$S = \{1, 2\}$$

and transition probability matrix

$$P = \begin{pmatrix} \alpha & 1 - \alpha \\ 1 - \beta & \beta \end{pmatrix}, \quad 0 < \alpha, \beta < 1.$$

Let $\boldsymbol{\pi} = (\pi_1, \pi_2)$ denote a stationary distribution. The stationary equations $\boldsymbol{\pi}P = \boldsymbol{\pi}$, together with the normalization condition, are

$$\begin{cases} \pi_1 = \pi_1\alpha + \pi_2(1 - \beta), \\ \pi_2 = \pi_1(1 - \alpha) + \pi_2\beta, \\ \pi_1 + \pi_2 = 1. \end{cases}$$

Solving these equations gives

$$\pi_1 = \frac{1 - \beta}{2 - \alpha - \beta}, \quad \pi_2 = \frac{1 - \alpha}{2 - \alpha - \beta}.$$

Since $0 < \alpha, \beta < 1$, the DTMC is finite, irreducible, and aperiodic (not forced into a fixed cycle). Therefore,

$$\lim_{n \rightarrow \infty} p_{ij}^{(n)} = \pi_j, \quad i, j \in S.$$

Hence, $\boldsymbol{\pi} = (\pi_1, \pi_2)$ is the limiting distribution, the stationary distribution, and the limiting occupancy distribution.

1.6 Rewards and Long-Run Averages

We can associate a cost or reward with each state of a DTMC. Let $c(i)$ denote the cost incurred in one period when the chain is in state i . That is, if $X_n = i$, then the cost at time n is $c(i)$.

Suppose the DTMC has a limiting occupancy distribution

$$\boldsymbol{\pi} = (\pi_j)_{j \in S}.$$

The long-run expected average cost per period, starting from state i , is defined by

$$\bar{c}(i) = \lim_{N \rightarrow \infty} \frac{1}{N+1} \mathbb{E} \left[\sum_{n=0}^N c(X_n) \mid X_0 = i \right].$$

Using the expected occupancy time $M_{ij}^{(N)}$, we can rewrite this quantity as

$$\bar{c}(i) = \lim_{N \rightarrow \infty} \sum_{j \in S} \frac{M_{ij}^{(N)}}{N+1} c(j).$$

If

$$\lim_{N \rightarrow \infty} \frac{M_{ij}^{(N)}}{N+1} = \pi_j,$$

then

$$\bar{c}(i) = \sum_{j \in S} \pi_j c(j).$$

Thus, the long-run expected average cost is the stationary weighted average of the one-period state costs.

Example 1.4: Service System

Consider a service system that operates as follows. At the beginning of each hour, the server checks whether there are any customers in the system. If there is at least one customer, the

server serves one customer for one hour. If there are no customers, the server remains idle for one hour and then checks again.

During each hour, the number of arriving customers is an independent random variable A with distribution

$$\mathbb{P}(A = 0) = 0.25, \quad \mathbb{P}(A = 1) = 0.5, \quad \mathbb{P}(A = 2) = 0.25.$$

The system capacity, including the customer in service, is 2. Customers who arrive when the system is full are lost.

What is the long-run proportion of time that the server is busy?

Let X_n denote the number of customers in the system at the beginning of the n th hour. Then $\{X_n, n \geq 0\}$ is a DTMC with state space

$$S = \{0, 1, 2\}.$$

The transition probability matrix is

$$P = \begin{pmatrix} 0.25 & 0.5 & 0.25 \\ 0.25 & 0.75 & 0 \\ 0 & 1 & 0 \end{pmatrix}.$$

The rows of P can be interpreted as follows:

- If $X_n = 0$, the server is idle during the hour. The next state is the number of arrivals during that hour, capped at capacity 2.
- If $X_n = 1$, the server serves one customer during the hour. If no customer arrives, the next state is 0. If one or two customers arrive, the next state is 1, because at most one newly arriving customer can wait for the next hour.
- If $X_n = 2$, one customer is in service and one customer is waiting. During the hour, any arriving customers are lost because the system is full. After service completion, the waiting customer remains in the system. Therefore, the next state is always 1.

Let $\boldsymbol{\pi} = (\pi_0, \pi_1, \pi_2)$ denote the stationary distribution. Solving

$$\boldsymbol{\pi}P = \boldsymbol{\pi}, \quad \pi_0 + \pi_1 + \pi_2 = 1,$$

gives

$$\pi_0 = \frac{4}{17}, \quad \pi_1 = \frac{12}{17}, \quad \pi_2 = \frac{1}{17}.$$

The server is busy during an hour if $X_n \geq 1$. Therefore, the long-run proportion of time that the server is busy is

$$\pi_1 + \pi_2 = \frac{12}{17} + \frac{1}{17} = \frac{13}{17}.$$

Example 1.5: Service System: Long-Run Average Profit

Consider the same service system. Suppose that every customer who receives service pays y , and the system incurs a penalty c for each lost customer. What is the long-run average profit?

The long-run average revenue is

$$y(\pi_1 + \pi_2) = y \left(\frac{12}{17} + \frac{1}{17} \right) = \frac{13}{17}y.$$

Next, compute the long-run expected number of lost customers per hour. If $X_n = 1$, one customer is lost only when $A = 2$, which occurs with probability 0.25. If $X_n = 2$, the system is full during the hour. In this case, one customer is lost when $A = 1$, and two customers are lost when $A = 2$. Hence, the expected number of lost customers in state 2 is

$$0.5 + 0.25 \times 2 = 1.$$

Thus, the long-run expected number of lost customers per hour is

$$0.25\pi_1 + \pi_2 = 0.25 \cdot \frac{12}{17} + \frac{1}{17} = \frac{3}{17} + \frac{1}{17} = \frac{4}{17}.$$

Therefore, the long-run average penalty is

$$\frac{4}{17}c.$$

Thus, the long-run average profit is

$$\frac{13}{17}y - \frac{4}{17}c.$$

Lecture 2 Exponential Clocks and Continuous Time Chains

Before introducing continuous time Markov chains, we review several basic facts about the exponential distribution.

2.1 Exponential Clocks

Definition

A nonnegative random variable X is called an *exponential random variable* with rate parameter $\lambda > 0$, written as

$$X \sim \exp(\lambda),$$

if its cumulative distribution function is

$$F_X(x) = \mathbb{P}(X \leq x) = \begin{cases} 0, & x < 0, \\ 1 - e^{-\lambda x}, & x \geq 0. \end{cases}$$

The probability density function of X is

$$f_X(x) = \begin{cases} 0, & x < 0, \\ \lambda e^{-\lambda x}, & x \geq 0. \end{cases}$$

The mean and variance are

$$\mathbb{E}[X] = \frac{1}{\lambda}, \quad \text{Var}(X) = \frac{1}{\lambda^2}.$$

Memoryless Property

Definition

A nonnegative random variable X is said to have the *memoryless property* if

$$\mathbb{P}(X > s + t \mid X > s) = \mathbb{P}(X > t), \quad s, t \geq 0.$$

If X represents the lifetime of a component, the memoryless property means that, conditional on the component having survived for s units of time, its remaining lifetime has the same distribution as a new component's lifetime.

Theorem

A continuous nonnegative random variable has the memoryless property if and only if it is an exponential random variable with rate parameter $\lambda > 0$ for some λ .

Hazard Rate

Let X be a nonnegative random variable with probability density function $f(\cdot)$ and cumulative distribution function $F(\cdot)$. The *hazard rate*, also called the *failure rate*, of X is defined as

$$r(x) = \frac{f(x)}{1 - F(x)}, \quad x \geq 0,$$

whenever $1 - F(x) > 0$.

For small $h > 0$, the conditional probability that X falls in the interval $(x, x + h]$, given that $X > x$, is approximately

$$\mathbb{P}(X \leq x + h \mid X > x) \approx hr(x).$$

Thus, $r(x)$ measures the instantaneous failure rate at age x , conditional on survival up to age x .

Theorem

A continuous nonnegative random variable has constant hazard rate

$$r(x) = \lambda$$

if and only if it is an exponential random variable with rate parameter $\lambda > 0$.

2.2 Competing Exponential Clocks

Let

$$X_i \sim \exp(\lambda_i), \quad 1 \leq i \leq k,$$

be independent exponential random variables. Define

$$Z = \min\{X_1, X_2, \dots, X_k\}$$

and let

$$\lambda = \sum_{i=1}^k \lambda_i.$$

Theorem

The minimum Z is exponentially distributed with rate

$$\lambda = \sum_{i=1}^k \lambda_i.$$

That is,

$$Z \sim \exp(\lambda).$$

Moreover, for each $1 \leq i \leq k$,

$$\mathbb{P}(Z = X_i) = \frac{\lambda_i}{\lambda}.$$

Finally, Z and the identity of the random variable that attains the minimum are independent.

Proof. For $t \geq 0$,

$$\mathbb{P}(Z > t) = \mathbb{P}(X_1 > t, X_2 > t, \dots, X_k > t).$$

By independence,

$$\mathbb{P}(Z > t) = \prod_{i=1}^k \mathbb{P}(X_i > t) = \prod_{i=1}^k e^{-\lambda_i t} = e^{-\left(\sum_{i=1}^k \lambda_i\right)t} = e^{-\lambda t}.$$

Therefore,

$$Z \sim \exp(\lambda).$$

Next, the probability that X_i is the first exponential clock to ring is proportional to its rate. More formally,

$$\mathbb{P}(Z = X_i) = \int_0^\infty \lambda_i e^{-\lambda_i t} \prod_{\ell \neq i} e^{-\lambda_\ell t} dt.$$

Thus,

$$\mathbb{P}(Z = X_i) = \int_0^\infty \lambda_i e^{-\lambda t} dt = \frac{\lambda_i}{\lambda}.$$

□

2.3 Poisson Arrivals

A Poisson process can be constructed from exponential interarrival times. Let $\{X_n, n \geq 1\}$ be a sequence of i.i.d. exponential random variables with rate parameter $\lambda > 0$. The random variable X_n represents the interarrival time between the $(n - 1)$ st event and the n th event.

Let

$$S_0 = 0, \quad S_n = \sum_{i=1}^n X_i, \quad n \geq 1.$$

Then S_n denotes the time of the n th event.

Define

$$N(t) = \max\{n \geq 0 : S_n \leq t\},$$

which is the number of events that have occurred by time t . Then

$$\{N(t), t \geq 0\}$$

is called a *counting process*. In this construction, $\{N(t), t \geq 0\}$ is a *Poisson process* with rate λ , denoted by

$$N(t) \sim PP(\lambda).$$

A sample path of a Poisson process is piecewise constant and right continuous. It increases by one at each event time $S_1, S_2, \dots$. Examples of Poisson processes include the number of births in a maternity ward, the number of customers arriving at a store, or the number of phone calls arriving at a call center over time.

Example 2.1: Kidney Transplant Allocation

Two individuals, A and B, both require kidney transplants. If they do not receive new kidneys, A will die after an exponential time with rate μ_A , and B will die after an exponential time with rate μ_B . New kidneys become available according to a Poisson process with rate λ . The first available kidney will go to A if A is alive at that time. If A is not alive but B is alive, the first available kidney will go to B. The next available kidney will go to B if B is still alive. What is the probability that A obtains a new kidney? What is the probability that B obtains a new kidney?

Let X_i denote the interarrival time between the $(i - 1)$ st and the i th kidney. Then

$$X_i \sim \exp(\lambda).$$

Let T_A and T_B denote the lifetimes of A and B without a transplant:

$$T_A \sim \exp(\mu_A), \quad T_B \sim \exp(\mu_B).$$

We assume that X_1, X_2, T_A, T_B are independent.

A obtains a kidney if the first kidney arrives before A dies. Therefore,

$$\mathbb{P}(\text{A obtains a kidney}) = \mathbb{P}(X_1 < T_A).$$

Using the minimum of exponentials property,

$$\mathbb{P}(X_1 < T_A) = \frac{\lambda}{\lambda + \mu_A}.$$

Thus,

$$\mathbb{P}(\text{A obtains a kidney}) = \frac{\lambda}{\lambda + \mu_A}.$$

Next, consider the probability that B obtains a kidney. There are two mutually exclusive cases.

First, A dies before both B and the first kidney arrival, and then the first kidney arrives before B dies. This event has probability

$$\frac{\mu_A}{\lambda + \mu_A + \mu_B} \cdot \frac{\lambda}{\lambda + \mu_B}.$$

Second, the first kidney arrives before either A or B dies, so A receives the first kidney. Then B receives the second kidney if the second kidney arrives before B dies. This event has probability

$$\frac{\lambda}{\lambda + \mu_A + \mu_B} \cdot \frac{\lambda}{\lambda + \mu_B}.$$

Therefore,

$$\mathbb{P}(\text{B obtains a kidney}) = \frac{\mu_A}{\lambda + \mu_A + \mu_B} \cdot \frac{\lambda}{\lambda + \mu_B} + \frac{\lambda}{\lambda + \mu_A + \mu_B} \cdot \frac{\lambda}{\lambda + \mu_B}.$$

Equivalently,

$$\mathbb{P}(\text{B obtains a kidney}) = \frac{\lambda(\mu_A + \lambda)}{(\lambda + \mu_A + \mu_B)(\lambda + \mu_B)}.$$

Thus,

$$\mathbb{P}(\text{B obtains a kidney}) = \frac{\mu_A}{\lambda + \mu_A + \mu_B} \cdot \frac{\lambda}{\lambda + \mu_B} + \frac{\lambda}{\lambda + \mu_A + \mu_B} \cdot \frac{\lambda}{\lambda + \mu_B}.$$

2.4 Continuous Time Markov Chains

A continuous time stochastic process

$$\{X(t), t \geq 0\}$$

is called a *continuous time Markov chain*, or *CTMC*, if it takes values in a countable state space S and satisfies the Markov property in continuous time. That is, for all $s, t \geq 0$ and states $i, j \in S$,

$$\mathbb{P}\{X(s+t) = j \mid X(s) = i, X(u) = x(u), 0 \leq u < s\} = \mathbb{P}\{X(s+t) = j \mid X(s) = i\},$$

whenever the conditional probabilities are well defined.

If, in addition,

$$\mathbb{P}\{X(s+t) = j \mid X(s) = i\}$$

is independent of the current time s , then the CTMC is said to be *time homogeneous*. For a time homogeneous CTMC, define

$$p_{ij}(t) = \mathbb{P}\{X(s+t) = j \mid X(s) = i\}, \quad i, j \in S, t \geq 0.$$

Because of time homogeneity, $p_{ij}(t)$ depends only on the elapsed time t , not on the starting time s .

2.5 Sojourn Times

Suppose a time homogeneous CTMC enters state i at time 0. If the process has already stayed in state i for s time units, then the Markov property implies that the probability it remains in state i for another t time units depends only on t , not on s . More precisely,

$$\mathbb{P}\{X(u) = i, s < u \leq s + t \mid X(u) = i, 0 \leq u \leq s\} = \mathbb{P}\{X(u) = i, 0 < u \leq t \mid X(0) = i\}.$$

Let τ_i denote the amount of time that the CTMC spends in state i before making a transition to a different state. Then

$$\mathbb{P}(\tau_i > s + t \mid \tau_i > s) = \mathbb{P}(\tau_i > t), \quad s, t \geq 0.$$

Thus, τ_i has the memoryless property. Therefore, τ_i is exponentially distributed.

This leads to an alternative characterization of a time homogeneous CTMC. Each time the process enters state $i \in S$:

1. the amount of time it spends in state i before moving to a different state is an exponential random variable with rate q_i ;
2. when the process leaves state i , it next enters state $j \neq i$ with probability p_{ij} , where

$$\sum_{j \neq i} p_{ij} = 1;$$

3. the sojourn time in state i and the next state visited are independent.

2.6 Generator Matrices

For $i \neq j$, define the *transition rate* from state i to state j by

$$q_{ij} = q_i p_{ij}.$$

The quantity q_{ij} represents the instantaneous rate at which the CTMC moves from state i to state j .

For technical reasons, define the diagonal entry by

$$q_{ii} = -q_i.$$

Equivalently,

$$q_{ii} = -\sum_{j \neq i} q_{ij}.$$

The matrix

$$Q = (q_{ij})_{i,j \in S}$$

is called the *generator matrix* of the CTMC. It plays the same role for CTMCs that the transition probability matrix P plays for DTMCs.

The off diagonal entries of Q are nonnegative:

$$q_{ij} \geq 0, \quad i \neq j,$$

and each row of Q sums to zero:

$$\sum_{j \in S} q_{ij} = 0, \quad i \in S.$$

Rate diagrams for CTMCs are drawn based on the off diagonal entries of Q . A positive entry q_{ij} corresponds to a directed edge from state i to state j , labeled by rate q_{ij} .

Example 2.2: Machine Maintenance

Consider a two state machine that can be either down or up. Let

$$S = \{0, 1\},$$

where state 0 means that the machine is down and state 1 means that the machine is up.

Suppose the machine stays operational for an exponential amount of time with rate λ . After it fails, it is repaired for an exponential amount of time with rate μ . Once the repair is completed, the machine becomes operational again.

The state of the machine is a CTMC because the sojourn time in each state is exponentially distributed, and the next state is determined by the current state. The generator matrix is

$$Q = \begin{pmatrix} -\mu & \mu \\ \lambda & -\lambda \end{pmatrix}.$$

Here, the transition rate from state 0 to state 1 is μ , and the transition rate from state 1 to state 0 is λ .

Lecture 3 Birth and Death Processes and Queues

3.1 Birth and Death Structure

A CTMC

$$\{X(t), t \geq 0\}$$

on the state space

$$S = \{0, 1, 2, \dots\}$$

is called a *birth-and-death process* if transitions are only allowed between neighboring states. That is, from state i , the process can move only to state $i + 1$ or state $i - 1$.

The transition rate from state i to state $i + 1$ is denoted by λ_i and is called the *birth rate* in state i . The transition rate from state i to state $i - 1$ is denoted by μ_i and is called the *death rate* in state i . Since death cannot occur when the process is in state 0, we define

$$\mu_0 = 0.$$

Thus, the nonzero transition rates are

$$q_{i,i+1} = \lambda_i, \quad i \geq 0,$$

and

$$q_{i,i-1} = \mu_i, \quad i \geq 1.$$

The diagonal entries of the generator matrix are

$$q_{ii} = -(\lambda_i + \mu_i), \quad i \geq 0,$$

where $\mu_0 = 0$.

Therefore, the generator matrix has the form

$$Q = \begin{pmatrix} -\lambda_0 & \lambda_0 & 0 & 0 & \cdots \\ \mu_1 & -(\lambda_1 + \mu_1) & \lambda_1 & 0 & \cdots \\ 0 & \mu_2 & -(\lambda_2 + \mu_2) & \lambda_2 & \cdots \\ 0 & 0 & \mu_3 & -(\lambda_3 + \mu_3) & \ddots \\ \vdots & \vdots & \vdots & \ddots & \ddots \end{pmatrix}.$$

When the process is in state i , it spends an exponential amount of time with rate

$$\lambda_i + \mu_i.$$

After that time, it jumps to state $i + 1$ with probability

$$\frac{\lambda_i}{\lambda_i + \mu_i},$$

or to state $i - 1$ with probability

$$\frac{\mu_i}{\lambda_i + \mu_i}.$$

For example, in a biological system, $X(t)$ may represent the number of organisms in a colony at time t . A transition from i to $i + 1$ represents a birth, and a transition from i to $i - 1$ represents a death.

3.2 Queueing Models

A queueing system describes how customers arrive, wait, receive service, and depart. A queueing model is usually specified by the following components.

- **Arrival process.** Customers arrive one by one, and the interarrival times are often assumed to be i.i.d. Common assumptions include:
 - M : exponential interarrival times, corresponding to a Poisson arrival process;
 - G : a general interarrival time distribution;
 - D : deterministic interarrival times.
- **Service times.** Service times are also often assumed to be i.i.d. They may be exponential, general, or deterministic.
- **Number of servers.** The system has s identical servers.
- **System capacity.** The system capacity is denoted by K . If no finite capacity is specified, the default assumption is $K = \infty$.
- **Service discipline.** The service discipline determines the order in which customers are served. Common examples include first-come, first-served (FCFS) and last-come, first-served (LCFS).

A standard notation for queueing systems is Kendall's notation:

$$A/B/s/K,$$

where A describes the interarrival time distribution, B describes the service time distribution, s is the number of servers, and K is the system capacity.

3.3 $M/M/1$ Queue

An $M/M/1$ queue is a single server queueing system with Poisson arrivals and exponential service times.

- Customers arrive according to a Poisson process with rate λ .
- Customers are served by a single server.
- The service times are i.i.d. exponential random variables with rate μ .
- The waiting room is unlimited.

Let $X(t)$ denote the number of customers in the system at time t , including the customer in service, if any. The state space is

$$S = \{0, 1, 2, \dots\}.$$

We ask whether

$$\{X(t), t \geq 0\}$$

is a CTMC.

When $X(t) = 0$, the system is empty. The next event must be an arrival, which occurs after an exponential time with rate λ . Thus, in state 0, the sojourn time is exponential with rate λ , and the process moves to state 1 with probability 1. Therefore,

$$p_{01} = 1, \quad q_{01} = \lambda, \quad q_{00} = -\lambda.$$

When $X(t) = i \geq 1$, both arrivals and service completions are possible. The arrival clock has rate λ , and the service completion clock has rate μ . Therefore, the time to the next event is exponential with rate

$$\lambda + \mu.$$

At the next event, the process moves to $i + 1$ if the arrival occurs first, and it moves to $i - 1$ if the service completion occurs first. By the minimum of exponentials property,

$$p_{i,i+1} = \frac{\lambda}{\lambda + \mu}, \quad p_{i,i-1} = \frac{\mu}{\lambda + \mu}.$$

The corresponding transition rates are

$$q_{i,i+1} = \lambda, \quad q_{i,i-1} = \mu, \quad q_{ii} = -(\lambda + \mu), \quad i \geq 1.$$

Thus,

$$\{X(t), t \geq 0\}$$

is a CTMC. More specifically, it is a birth-and-death process with birth rates

$$\lambda_i = \lambda, \quad i \geq 0,$$

and death rates

$$\mu_i = \mu, \quad i \geq 1,$$

with $\mu_0 = 0$.

3.4 Transient Probabilities

We now consider the transient behavior of a time-homogeneous CTMC

$$\{X(t), t \geq 0\}.$$

Let

$$a_i = \mathbb{P}\{X(0) = i\}, \quad i \in S.$$

The vector

$$\mathbf{a} = (a_i)_{i \in S}$$

is called the *initial distribution* of the CTMC.

For $t \geq 0$, define

$$p_{ij}(t) = \mathbb{P}\{X(t) = j \mid X(0) = i\}, \quad i, j \in S.$$

This is the transition probability from state i to state j over elapsed time t .

Let $V_j(t)$ denote the amount of time the CTMC spends in state j over the time interval $[0, t]$. Equivalently,

$$V_j(t) = \int_0^t \mathbf{1}\{X(u) = j\} du.$$

Define

$$M_{ij}(t) = \mathbb{E}[V_j(t) \mid X(0) = i],$$

which is the expected occupancy time of state j up to time t , starting from state i .

Theorem

For any states $i, j \in S$ and any $t \geq 0$,

$$M_{ij}(t) = \int_0^t p_{ij}(u) du.$$

Proof. Using

$$V_j(t) = \int_0^t \mathbf{1}\{X(u) = j\} du,$$

we have

$$M_{ij}(t) = \mathbb{E} \left[\int_0^t \mathbf{1}\{X(u) = j\} du \mid X(0) = i \right].$$

Interchanging expectation and integration gives

$$M_{ij}(t) = \int_0^t \mathbb{E} [\mathbf{1}\{X(u) = j\} \mid X(0) = i] du.$$

Since

$$\mathbb{E} [\mathbf{1}\{X(u) = j\} \mid X(0) = i] = \mathbb{P}\{X(u) = j \mid X(0) = i\} = p_{ij}(u),$$

it follows that

$$M_{ij}(t) = \int_0^t p_{ij}(u) du.$$

□

3.5 Stationary Behavior and Applications

For an irreducible and positive recurrent CTMC, the transition probabilities converge to a limiting distribution. That is,

$$\lim_{t \rightarrow \infty} p_{ij}(t) = p_j > 0, \quad i, j \in S.$$

The vector

$$\mathbf{p} = (p_j)_{j \in S}$$

is the unique solution to the balance equations

$$p_j q_j = \sum_{i \neq j} p_i q_{ij}, \quad j \in S,$$

together with the normalizing equation

$$\sum_{j \in S} p_j = 1.$$

Equivalently, if $Q = (q_{ij})_{i,j \in S}$ is the generator matrix, then

$$\mathbf{p}Q = \mathbf{0}, \quad \sum_{j \in S} p_j = 1.$$

The vector $\mathbf{p}$ is called the *limiting distribution*, or *stationary distribution*. It is also the *stationary distribution* and the *limiting occupancy distribution* of the CTMC.

In particular,

$$\lim_{t \rightarrow \infty} \frac{1}{t} M_{ij}(t) = p_j.$$

Thus, in the long run, the fraction of time that the CTMC spends in state j is p_j .

Example 3.1: Machine Maintenance

Consider a two state machine that can be either down or up. Let

$$S = \{0, 1\},$$

where state 0 means that the machine is down and state 1 means that the machine is up.

Suppose the machine stays operational for an exponential amount of time with rate λ . After it fails, it is repaired for an exponential amount of time with rate μ . Once the repair is completed, the machine becomes operational again.

The state of the machine is a CTMC. Its generator matrix is

$$Q = \begin{pmatrix} -\mu & \mu \\ \lambda & -\lambda \end{pmatrix}.$$

Here, the transition rate from state 0 to state 1 is μ , and the transition rate from state 1 to state 0 is λ .

Let

$$\mathbf{p} = (p_0, p_1)$$

denote the stationary distribution. The balance equations are

$$\mu p_0 = \lambda p_1,$$

together with

$$p_0 + p_1 = 1.$$

Solving these equations gives

$$p_0 = \frac{\lambda}{\lambda + \mu}, \quad p_1 = \frac{\mu}{\lambda + \mu}.$$

Thus, in the long run, the machine is down for a fraction

$$\frac{\lambda}{\lambda + \mu}$$

of the time and up for a fraction

$$\frac{\mu}{\lambda + \mu}$$

of the time.

3.6 Rewards, Costs, and Net Profit

We can associate a cost or reward rate with each state of a CTMC. Let $c(i)$ denote the cost rate incurred per unit time when the CTMC is in state i . That is, if $X(t) = i$, then the process incurs cost at rate $c(i)$.

Suppose the CTMC has a limiting distribution

$$\mathbf{p} = (p_j)_{j \in S}.$$

The long run expected average cost per unit time, starting from state i , is defined by

$$\bar{c}(i) = \lim_{t \rightarrow \infty} \frac{1}{t} \mathbb{E} \left[\int_0^t c(X(u)) du \mid X(0) = i \right].$$

Using the expected occupancy time $M_{ij}(t)$, we can rewrite this quantity as

$$\bar{c}(i) = \lim_{t \rightarrow \infty} \sum_{j \in S} \frac{M_{ij}(t)}{t} c(j).$$

If

$$\lim_{t \rightarrow \infty} \frac{M_{ij}(t)}{t} = p_j,$$

then

$$\bar{c}(i) = \sum_{j \in S} p_j c(j).$$

Thus, the long run expected average cost per unit time is the stationary weighted average of the state cost rates.

Example 3.2: $M/M/1$ Queue: Long Run Average Net Profit

Consider an $M/M/1$ queue. Customers arrive according to a Poisson process with rate λ . Customers are served by a single server, and service times are i.i.d. exponential random variables with rate μ . Assume

$$\lambda < \mu,$$

so that the CTMC is positive recurrent.

Suppose each customer pays an entry fee d upon joining the queue. Also suppose the system incurs a holding cost c per customer per unit time, including customers waiting in queue and the customer receiving service. We want to compute the long run average net profit.

Let

$$X(t)$$

denote the number of customers in the system at time t . Then

$$S = \{0, 1, 2, \dots\}.$$

This is a birth and death process with birth rates

$$\lambda_i = \lambda, \quad i \geq 0,$$

and death rates

$$\mu_i = \mu, \quad i \geq 1.$$

Let

$$\mathbf{p} = (p_0, p_1, p_2, \dots)$$

denote the stationary distribution. The balance equations are

$$\lambda p_0 = \mu p_1,$$

and, for $i \geq 1$,

$$(\lambda + \mu)p_i = \lambda p_{i-1} + \mu p_{i+1}.$$

From the first balance equation,

$$p_1 = \frac{\lambda}{\mu} p_0.$$

Similarly, the balance equations imply

$$p_i = \left(\frac{\lambda}{\mu}\right)^i p_0, \quad i \geq 0.$$

Let

$$\rho = \frac{\lambda}{\mu},$$

which is called the *traffic intensity* of the queue. Since $\lambda < \mu$, we have $\rho < 1$. Using the normalizing condition,

$$\sum_{i=0}^{\infty} p_i = 1,$$

we obtain

$$p_i = (1 - \rho)\rho^i, \quad i \geq 0.$$

The holding cost rate in state j is

$$c(j) = cj, \quad j \geq 0,$$

because there are j customers in the system. Therefore, the long run average cost per unit time is

$$\sum_{j=0}^{\infty} p_j c(j) = \sum_{j=0}^{\infty} (1 - \rho)\rho^j cj.$$

Since

$$\sum_{j=0}^{\infty} j\rho^j = \frac{\rho}{(1 - \rho)^2},$$

we get

$$\sum_{j=0}^{\infty} (1 - \rho)\rho^j cj = c(1 - \rho) \frac{\rho}{(1 - \rho)^2} = \frac{c\rho}{1 - \rho}.$$

The long run average reward per unit time is

$$\lambda d,$$

because customers enter the system at rate λ , and each customer pays d .

Hence, the long run average net profit per unit time is

$$\lambda d - \frac{c\rho}{1 - \rho}.$$

Lecture 4 Finite Horizon MDPs and Backward Induction

4.1 States, Actions, and Rewards

Markov decision processes extend Markov chains by allowing a decision maker to choose actions over time. Classical MDP models include:

- finite horizon problems;
- infinite horizon problems.

MDPs have many applications, including inventory control, revenue management, queueing control, healthcare decision making, sports strategy, and other sequential decision making problems.

In this course, we will learn how to apply MDPs, identify model components, solve models using exact or approximate methods, and interpret the resulting solutions.

Feasible actions

We consider a discrete time decision process indexed by

$$n = 0, 1, 2, \dots$$

Let S_n denote the state of the system at time n . We assume

$$S_n \in S,$$

where S is called the *state space*. For example, one may have

$$S = \{0, 1, 2, \dots\}.$$

At each time n , the decision maker chooses an action A_n . We write

$$A_n \in A,$$

where A is called the *action space*. For example, one may have

$$A = \{1, 2, \dots, K\}.$$

In many MDP models, the set of feasible actions depends on the current state. In that case, if $S_n = i$, then the action must satisfy

$$A_n \in A(i),$$

where $A(i)$ is the set of admissible actions in state i .

4.2 Histories and Policies

At time n , the history of the process is

$$H_n = (S_0, A_0, S_1, A_1, \dots, S_{n-1}, A_{n-1}).$$

Together with the current state S_n , the information available at time n is

$$(H_n, S_n).$$

Based on this information, the decision maker chooses the action A_n .

Definition

A discrete time controlled stochastic process is called a *Markov decision process*, or *MDP*, if the distribution of the next state depends on the past only through the current state and the current action. That is, for all states $i, j \in S$, actions $a \in A(i)$, and histories H_n ,

$$\mathbb{P}(S_{n+1} = j \mid S_n = i, A_n = a, H_n) = \mathbb{P}(S_{n+1} = j \mid S_n = i, A_n = a).$$

The quantity

$$p_{ij}(a) = \mathbb{P}(S_{n+1} = j \mid S_n = i, A_n = a)$$

is called the *transition probability* from state i to state j under action a .

If the transition probabilities do not depend on time n , then the MDP is called *time homogeneous*. In this case, for each state $i \in S$ and action $a \in A(i)$, the transition probabilities satisfy

$$\sum_{j \in S} p_{ij}(a) = 1.$$

Thus, an MDP model is specified by the following main components:

- a state space S ;
- an action space A , or state dependent action sets $A(i)$;
- transition probabilities $p_{ij}(a)$;
- a cost or reward structure;
- an objective criterion, such as finite horizon total reward, infinite horizon discounted reward, or long run average reward.

4.3 Backward Recursion

We now focus on finite horizon MDPs. In a finite horizon problem, decisions are made over a finite number of periods. Let N denote the finite horizon.

Decisions are made at times $n = 0, \dots, N-1$. If the final state is $S_N = i$, a terminal reward $g_{\text{term}}(i)$ is received. The objective is

$$\max_{\pi} \mathbb{E}^{\pi} \left[\sum_{n=0}^{N-1} G_{n+1} + g_{\text{term}}(S_N) \mid S_0 = i \right],$$

where G_{n+1} is the reward observed after the action at time n . Its conditional mean is $g(S_n, A_n)$.

Finite horizon MDPs are usually solved by *backward recursion*. The key idea is to start from the terminal time and work backward.

Define

$$V_m(i)$$

as the maximum expected total reward when there are m decisions left and the current state is i . In particular,

$$V_0(i) = g_{\text{term}}(i),$$

because when no decisions are left, the only remaining reward is the terminal reward.

When the horizon is N , the value

$$V_N(i)$$

is the maximum expected total reward when the process starts in state i at time 0.

4.4 Bellman Equation

Suppose there are $m \geq 1$ decisions left and the current state is i . If the decision maker chooses action $a \in A(i)$, then:

- the immediate expected reward is $g(i, a)$;
- the next state is j with probability $p_{ij}(a)$;
- after moving to state j , there are $m - 1$ decisions left, so the best expected reward from then on is $V_{m-1}(j)$.

Therefore, the maximum expected total reward in state i with m decisions left satisfies the Bellman equation

$$V_m(i) = \max_{a \in A(i)} \left\{ g(i, a) + \sum_{j \in S} p_{ij}(a) V_{m-1}(j) \right\}, \quad m = 1, 2, \dots, N.$$

The boundary condition is

$$V_0(i) = g_{\text{term}}(i), \quad i \in S.$$

4.5 Recovering an Optimal Policy

The function $V_m(i)$ is the maximum expected reward when the current state is i and m decisions remain. Restarting the clock at the current state gives the equivalent expression

$$V_m(i) = \max_{\pi} \mathbb{E}_i^{\pi} \left[\sum_{t=0}^{m-1} G_{t+1} + g_{\text{term}}(S_m) \right].$$

The policy in this expression may depend on the state and on the number of decisions remaining.

For each $m = 1, 2, \dots, N$, an optimal decision rule is a function

$$d_m : S \rightarrow A$$

such that

$$d_m(i) \in \arg \max_{a \in A(i)} \left\{ g(i, a) + \sum_{j \in S} p_{ij}(a) V_{m-1}(j) \right\}.$$

An optimal finite horizon policy is then given by

$$\pi^* = (d_N, d_{N-1}, \dots, d_1).$$

Thus, the optimal action can depend on both the current state and the number of decisions remaining. For finite horizon MDPs, there exists a deterministic Markovian policy that is optimal.

4.6 Stopping as a Finite Horizon MDP

Example 4.1: American put option

An American put option on a stock with strike price K and expiry date T is a financial derivative that gives the owner the right, but not the obligation, to sell one share of the stock at price K at any time at or before T .

If the owner does not exercise the option by time T , the option expires. If the owner exercises the option at time $n \leq T$, and the stock price is X_n , then the immediate payoff is

$$K - X_n.$$

Clearly, exercising is attractive only when $K \geq X_n$. The question is: what is the optimal exercise policy that maximizes the owner's expected payoff?

Suppose the stock price process

$$\{X_n, n \geq 0\}$$

is a DTMC on

$$S = \{0, 1, 2, \dots\}$$

with transition probability matrix

$$P = (p_{ij})_{i,j \in S}.$$

The system is in state i if the current stock price is i and the option has not yet been exercised.

Let $V_m(i)$ denote the maximum expected payoff from the option when there are m days left and the current stock price is i . Equivalently, the current state is

$$X_{T-m} = i.$$

At maturity, there are no future decisions left. Therefore,

$$V_0(i) = \max\{K - i, 0\}, \quad i \in S.$$

For $1 \leq m \leq T$, the owner has two choices:

- exercise immediately and receive payoff $K - i$;
- wait one more period, in which case the expected continuation value is

$$\sum_{j \in S} p_{ij} V_{m-1}(j).$$

Thus, the Bellman equation is

$$V_m(i) = \max \left\{ K - i, \sum_{j \in S} p_{ij} V_{m-1}(j) \right\}, \quad i \in S, \quad 1 \leq m \leq T.$$

The optimal policy is to exercise the option when the immediate exercise value is at least as large as the continuation value:

$$K - i \geq \sum_{j \in S} p_{ij} V_{m-1}(j).$$

Otherwise, the owner should wait.

Example 4.2: American put option: numerical calculation

Suppose

$$S = \{0, 1, 2, \dots, 20\}, \quad K = 10, \quad T = 90.$$

The transition probabilities are

$$\begin{aligned} p_{00} &= 0.45, \\ p_{i,i+1} &= 0.55, \quad 0 \leq i \leq 19, \\ p_{i,i-1} &= 0.45, \quad 1 \leq i \leq 20, \end{aligned}$$

and

$$p_{20,20} = 0.55.$$

The optimality equations become

$$V_0(i) = \max\{10 - i, 0\}, \quad 0 \leq i \leq 20,$$

and, for $1 \leq m \leq 90$,

$$V_m(i) = \max \left\{ 10 - i, \sum_{j=0}^{20} p_{ij} V_{m-1}(j) \right\}, \quad 0 \leq i \leq 20.$$

In this numerical example, the optimal policy has a threshold structure. For each $m \geq 0$, there exists a threshold i_m such that, when there are m more periods to go, it is optimal to exercise the option if

$$i \leq i_m,$$

and it is optimal to wait if

$$i > i_m.$$

For this numerical example, the thresholds are:

$$i_0 = 10,$$

$$i_1 = 9,$$

$$i_m = 8, \quad 2 \leq m \leq 4,$$

$$i_m = 7, \quad 5 \leq m \leq 15,$$

$$i_m = 6, \quad 16 \leq m \leq 64,$$

and

$$i_m = 5, \quad 65 \leq m \leq 90.$$

A natural question is whether this threshold structure also holds for the general problem. To answer this question, we introduce a monotonicity condition on the transition probability matrix.

Definition

A matrix

$$P = (p_{ij})_{i,j \in S}$$

is called a *stochastic matrix* if

$$p_{ij} \geq 0$$

and

$$\sum_{j \in S} p_{ij} = 1$$

for every row i .

Definition

A stochastic matrix P is called *stochastically monotone* if

$$\sum_{j=k}^{\infty} p_{ij}$$

is an increasing function of i for every $k \geq 0$.

If P is the transition probability matrix of a DTMC, then P is stochastically monotone if and only if

$$\mathbb{P}(X_{n+1} \geq k \mid X_n = i)$$

is increasing in i , for any fixed k .

A vector

$$h = (h_0, h_1, h_2, \dots)$$

is nondecreasing if

$$h_{i+1} \geq h_i, \quad i \geq 0.$$

Theorem

A stochastic matrix P is stochastically monotone if and only if

$$Ph$$

is nondecreasing for every nondecreasing vector h .

Now suppose that $\{X_n, n \geq 0\}$ is a DTMC on

$$\{0, 1, 2, \dots\}$$

with a stochastically monotone transition matrix P . Define

$$f(i) = \mathbb{E}(X_{n+1} \mid X_n = i) = \sum_{j=0}^{\infty} j p_{ij}.$$

Using the identity

$$\sum_{j=0}^{\infty} j p_{ij} = \sum_{k=1}^{\infty} \sum_{j=k}^{\infty} p_{ij},$$

we see that $f(i)$ is an increasing function of i .

Define the mean drift in state i by

$$d(i) = \mathbb{E}(X_{n+1} - X_n \mid X_n = i) = f(i) - i.$$

Definition

A stochastic matrix P is called *drift constrained* if $d(i)$ is a decreasing function of i for $i \geq 0$.

Theorem

Suppose P is stochastically monotone and drift constrained. Then

$$V_m(i) + i$$

is an increasing function of i for every $m \geq 0$.

Proof. We prove the result by induction on m .

For $m = 0$,

$$V_0(i) + i = \max\{K - i, 0\} + i = \max\{K, i\},$$

which is increasing in i .

Now suppose $V_{m-1}(i) + i$ is increasing in i . For $m \geq 1$,

$$V_m(i) = \max \left\{ K - i, \sum_{j=0}^{\infty} p_{ij} V_{m-1}(j) \right\}.$$

Therefore,

$$V_m(i) + i = \max \left\{ K, \sum_{j=0}^{\infty} p_{ij} V_{m-1}(j) + i \right\}.$$

Rewrite the second term as

$$\sum_{j=0}^{\infty} p_{ij} V_{m-1}(j) + i = \sum_{j=0}^{\infty} p_{ij} (V_{m-1}(j) + j) + i - \sum_{j=0}^{\infty} j p_{ij}.$$

Since

$$d(i) = \sum_{j=0}^{\infty} j p_{ij} - i,$$

we have

$$i - \sum_{j=0}^{\infty} j p_{ij} = -d(i).$$

Thus,

$$V_m(i) + i = \max \left\{ K, \sum_{j=0}^{\infty} p_{ij} (V_{m-1}(j) + j) - d(i) \right\}.$$

By the induction hypothesis, $V_{m-1}(j) + j$ is increasing in j . Since P is stochastically monotone,

$$\sum_{j=0}^{\infty} p_{ij} (V_{m-1}(j) + j)$$

is increasing in i . Since P is drift constrained, $d(i)$ is decreasing in i , so $-d(i)$ is increasing in i . Therefore,

$$\sum_{j=0}^{\infty} p_{ij} (V_{m-1}(j) + j) - d(i)$$

is increasing in i . Taking the maximum with the constant K preserves monotonicity. Hence $V_m(i) + i$ is increasing in i . $\square$

Define

$$g_m(i) = \sum_{j=0}^{\infty} p_{ij} (V_{m-1}(j) + j) - d(i).$$

Then

$$V_m(i) + i = \max\{K, g_m(i)\}.$$

Under the assumptions of stochastic monotonicity and drift constraint, $g_m(i)$ is increasing in i .

Let

$$i_m = \max\{i \geq 0 : K \geq g_m(i)\}.$$

Then the optimal decision when the price is i and there are m more periods to go is:

$$\text{exercise if } i \leq i_m,$$

and

$$\text{wait if } i > i_m.$$

Thus, the optimal policy has a threshold structure. In particular,

$$i_0 = K.$$

Lecture 5 Finite Horizon Resource and Reliability Models

5.1 State Balance and One Period Cost

Example 5.1: Inventory control

Consider a retailer that stocks a single item. Let S_n denote the inventory level at the beginning of day

$$n \in \{1, 2, \dots, N\}.$$

If $S_n > 0$, then S_n represents the amount of inventory on hand. If $S_n < 0$, then $-S_n$ represents the backorder level.

After observing S_n , the retailer orders an amount

$$A_n \geq 0$$

from the supplier. The order is delivered immediately, so there is no lead time. Thus, after ordering, the inventory level increases to

$$S_n + A_n.$$

Let D_n denote the demand during day n . Then the inventory level at the beginning of the next day is

$$S_{n+1} = S_n + A_n - D_n.$$

Assume that

$$\{D_n, n \geq 1\}$$

is a sequence of i.i.d. continuous nonnegative random variables with common cumulative distribution function F , probability density function f , and mean

$$\tau = \mathbb{E}[D_n].$$

The process

$$\{(S_n, A_n), n \geq 1\}$$

is an MDP. The state is the current inventory level S_n , and the action is the order quantity A_n .

The retailer incurs two types of costs:

- *holding cost*: each unit of inventory on hand at the end of the day costs h dollars;
- *backordering cost*: each unit of backorder at the end of the day costs p dollars.

The cost on day n is

$$C_n = h \max(S_n + A_n - D_n, 0) - p \min(S_n + A_n - D_n, 0).$$

The second term is positive because $\min(S_n + A_n - D_n, 0) \leq 0$, so

$$-p \min(S_n + A_n - D_n, 0)$$

is the backordering cost.

For a given state x and action a , the one period expected cost is

$$c(x, a) = \mathbb{E}[C_n \mid S_n = x, A_n = a].$$

Thus,

$$c(x, a) = h\mathbb{E}[\max(x + a - D_n, 0)] - p\mathbb{E}[\min(x + a - D_n, 0)].$$

Let

$$y = x + a$$

denote the inventory level immediately after ordering. We can write the expected cost as a function of y . If $y < 0$, then $y - D_n < 0$ because $D_n \geq 0$. Therefore,

$$c(y) = -p\mathbb{E}(y - D_n) = p\tau - py, \quad y < 0.$$

If $y \geq 0$, then

$$\mathbb{E}[\max(y - D_n, 0)] = \int_0^y (y - u)f(u) du,$$

and

$$-\mathbb{E}[\min(y - D_n, 0)] = \mathbb{E}[(D_n - y)^+] = \int_y^\infty (u - y)f(u) du.$$

Hence,

$$c(y) = h \int_0^y (y - u)f(u) du + p \int_y^\infty (u - y)f(u) du.$$

An equivalent expression is

$$c(y) = p\tau + hy - (h + p) \int_0^y (1 - F(u)) du, \quad y \geq 0.$$

Therefore,

$$c(x, a) = c(x + a) = \begin{cases} p\tau - p(x + a), & x + a < 0, \\ p\tau + h(x + a) - (h + p) \int_0^{x+a} (1 - F(u)) du, & x + a \geq 0. \end{cases}$$

5.2 Inventory Recursion and Order Up To Decisions

Let $V_m(x)$ denote the minimum expected total cost starting with inventory level x and m more decisions to go. The boundary condition is

$$V_0(x) = 0.$$

For $m = 1, 2, \dots, N$, the Bellman equation is

$$V_m(x) = \min_{a \geq 0} \{c(x, a) + \mathbb{E}[V_{m-1}(x + a - D)]\}.$$

Because $c(x, a)$ is a function of $x + a$, it is convenient to use

$$y = x + a$$

as the decision variable. The quantity y is called the *order up to level*. Since $a \geq 0$, the feasible values of y satisfy

$$y \geq x.$$

Thus, the Bellman equation can be rewritten as

$$V_m(x) = \min_{y \geq x} \{c(y) + \mathbb{E}[V_{m-1}(y - D)]\}, \quad m = 1, 2, \dots, N.$$

5.3 Convexity and Base Stock Structure

Now consider the case $m = 1$. Since $V_0(x) = 0$, we have

$$V_1(x) = \min_{y \geq x} c(y).$$

Suppose $c(y)$ is minimized at

$$y^*.$$

Then the optimal decision with one period to go is:

$$a_1^*(x) = 0, \quad \text{if } x \geq y^*,$$

and

$$a_1^*(x) = y^* - x, \quad \text{if } x < y^*.$$

This is called an *order up to policy*, and y^* is called the *base stock level*.

The value function for one period to go is

$$V_1(x) = \begin{cases} c(y^*), & x < y^*, \\ c(x), & x \geq y^*. \end{cases}$$

Under the assumptions above, $V_1(x)$ is convex and differentiable in x .

For two periods to go, the Bellman equation is

$$V_2(x) = \min_{y \geq x} \{c(y) + \mathbb{E}[V_1(y - D)]\}.$$

Since expectation preserves convexity,

$$\mathbb{E}[V_1(y - D)]$$

is convex in y . Therefore,

$$c(y) + \mathbb{E}[V_1(y - D)]$$

is also convex in y .

In this model, the same base stock level y^* remains optimal. Hence,

$$V_2(x) = \begin{cases} 2c(y^*), & x < y^*, \\ c(x) + \mathbb{E}[V_1(x - D)], & x \geq y^*. \end{cases}$$

More generally, by induction, the order up to policy with base stock level y^* is optimal when there are m periods to go, for all

$$m = 1, 2, \dots, N.$$

That is, the optimal action is

$$a_m^*(x) = \begin{cases} y^* - x, & x < y^*, \\ 0, & x \geq y^*. \end{cases}$$

5.4 Reliability States and Replacement Actions

Example 5.2: Machine maintenance

Suppose a manufacturing process requires continuous use of a machine. Let

$$S_n \in S = \{0, 1, 2, \dots\}$$

denote the state of the machine at time n . State 0 represents a new machine. Larger state values represent worse machine conditions.

Let $w(i)$ denote the revenue earned in one period when the machine is in state i . We assume that $w(i)$ is a decreasing function of i . That is, as the machine condition worsens, the revenue from using the machine decreases.

At each time n , the decision maker chooses an action

$$A_n \in \{0, 1\}.$$

The two actions are:

- $A_n = 0$: do nothing;
- $A_n = 1$: replace the machine.

If the machine is replaced, the replacement takes one period and the new machine starts in state 0. Let

$$c$$

denote the replacement cost.

If the decision maker does nothing, then the machine condition evolves according to the transition probabilities

$$\mathbb{P}(S_{n+1} = j \mid S_n = i, A_n = 0) = p_{ij}.$$

If the decision maker replaces the machine, then the next state is 0 with probability one:

$$\mathbb{P}(S_{n+1} = 0 \mid S_n = i, A_n = 1) = 1,$$

and

$$\mathbb{P}(S_{n+1} = j \mid S_n = i, A_n = 1) = 0, \quad j \neq 0.$$

The one period reward is

$$g(i, 0) = w(i), \quad g(i, 1) = -c.$$

Thus, doing nothing earns revenue $w(i)$, whereas replacing the machine incurs cost c .

The controlled process

$$\{(S_n, A_n), n \geq 0\}$$

is an MDP. Suppose the system starts with a new machine:

$$S_0 = 0.$$

Assume there is no terminal reward:

$$g_{\text{term}}(i) = 0, \quad i \in S.$$

Let $V_m(i)$ denote the maximum expected total net revenue over the remaining m periods when the current state is i . The boundary condition is

$$V_0(i) = g_{\text{term}}(i) = 0, \quad i \in S.$$

For $m \geq 1$, the Bellman equation is

$$V_m(i) = \max \left\{ V_{m-1}(0) - c, w(i) + \sum_{j \in S} p_{ij} V_{m-1}(j) \right\}.$$

The first term corresponds to replacing the machine. The second term corresponds to doing nothing.

5.5 Computing the Backward Recursion

Example 5.3: Machine maintenance numerical calculation

Suppose

$$c = 10, \quad S = \{0, 1, 2, 3, 4\}.$$

The transition probability matrix under the “do nothing” action is

$$P = \begin{pmatrix} 0.9 & 0.1 & 0 & 0 & 0 \\ 0 & 0.9 & 0.1 & 0 & 0 \\ 0 & 0 & 0.9 & 0.1 & 0 \\ 0 & 0 & 0 & 0.9 & 0.1 \\ 0 & 0 & 0 & 0 & 1 \end{pmatrix}.$$

The one period revenue function is

$$w(i) = 10 - 2i, \quad i = 0, 1, 2, 3,$$

and

$$w(4) = 0.$$

We want to compute

$$V_{10}(0),$$

the optimal expected net revenue over 10 days starting with a new machine.

For one period to go,

$$V_1(i) = \max\{-c, w(i)\}.$$

Since $c = 10$ and $w(i) \geq 0$ for all $i \in S$, we have

$$V_1(i) = w(i).$$

That is,

$$V_1(i) = \begin{cases} 10 - 2i, & i = 0, 1, 2, 3, \\ 0, & i = 4. \end{cases}$$

Thus, the optimal action with one period to go is to do nothing in every state:

$$d_1(i) = 0, \quad i \in S.$$

For two periods to go,

$$V_2(0) = \max\{10 - 10, 10 + 0.9(10) + 0.1(8)\} = 19.8,$$

$$V_2(1) = \max\{10 - 10, 8 + 0.9(8) + 0.1(6)\} = 15.8,$$

$$V_2(2) = \max\{10 - 10, 6 + 0.9(6) + 0.1(4)\} = 11.8,$$

$$V_2(3) = \max\{10 - 10, 4 + 0.9(4) + 0.1(0)\} = 7.6,$$

and

$$V_2(4) = \max\{10 - 10, 0\} = 0.$$

Therefore, the optimal action with two periods to go is still to do nothing in every state:

$$d_2(i) = 0, \quad i \in S.$$

For three periods to go,

$$V_3(0) = \max\{19.8 - 10, 10 + 0.9(19.8) + 0.1(15.8)\} = 29.4,$$

$$V_3(1) = \max\{19.8 - 10, 8 + 0.9(15.8) + 0.1(11.8)\} = 23.4,$$

$$V_3(2) = \max\{19.8 - 10, 6 + 0.9(11.8) + 0.1(7.6)\} = 17.38,$$

$$V_3(3) = \max\{19.8 - 10, 4 + 0.9(7.6) + 0.1(0)\} = 10.84,$$

and

$$V_3(4) = \max\{19.8 - 10, 0\} = 9.8.$$

Thus, the optimal decision rule with three periods to go is

$$d_3(i) = \begin{cases} 0, & i = 0, 1, 2, 3, \\ 1, & i = 4. \end{cases}$$

That is, with three periods to go, it is optimal to replace the machine only in state 4.

The optimal policy has a threshold structure. For each $m \geq 0$, let κ_m denote the threshold: do nothing if $i \leq \kappa_m$, and replace the machine if $i > \kappa_m$. In the numerical example,

$$\kappa_1 = \kappa_2 = 4, \quad \kappa_3 = 3.$$

5.6 Monotonicity and Threshold Policies

Theorem

Suppose the transition matrix P is stochastically monotone and the net revenue vector

$$w = (w(i))_{i \in S}$$

is nonincreasing. Then $V_m(\cdot)$ is nonincreasing for all $m \geq 0$.

Proof. We prove the result by induction on m .

For $m = 0$,

$$V_0(i) = 0, \quad i \in S,$$

so $V_0(\cdot)$ is nonincreasing.

Suppose $V_m(\cdot)$ is nonincreasing. Since P is stochastically monotone, PV_m is also nonincreasing. Since w is nonincreasing, the vector

$$w + PV_m$$

is nonincreasing.

The Bellman equation can be written as

$$V_{m+1} = \max\{V_m(0) - c, w + PV_m\},$$

where the maximum is taken componentwise. The first term $V_m(0) - c$ is a constant vector, and the second term is nonincreasing. Therefore, $V_{m+1}(\cdot)$ is nonincreasing. $\square$

Now assume that P is stochastically monotone and w is nonincreasing. Then the continuation value

$$w(i) + (PV_{m-1})(i)$$

is nonincreasing in i , whereas the replacement value

$$V_{m-1}(0) - c$$

does not depend on i . Therefore, there is a threshold κ_m such that

$$w(i) + (PV_{m-1})(i) \geq V_{m-1}(0) - c, \quad i \leq \kappa_m,$$

and the reverse strict inequality holds for $i > \kappa_m$. Hence the optimal policy does nothing for $i \leq \kappa_m$ and replaces the machine for $i > \kappa_m$.

Lecture 6 Finite Horizon Optimal Stopping

6.1 Stopping Decisions and Sufficient States

Example 6.1: Secretary problem

A firm wants to hire one person for a job. There are N prospective candidates, and the firm interviews them one by one.

After each candidate is interviewed, the firm must decide whether to hire this candidate. If the firm makes an offer, denoted by $a = 1$, the candidate accepts it. If the firm does not make an offer, denoted by $a = 0$, then the candidate is no longer available.

The firm can rank the candidates it has interviewed so far, but it does not know the rankings of candidates who have not yet been interviewed. The objective is to maximize the probability of hiring the best candidate among all N prospective candidates.

6.2 Bellman Recursion for Sequential Selection

Assume that the candidates are interviewed in a completely random order. That is, all possible permutations of their true rankings are equally likely.

Let X_n denote the state of the system after interviewing the first n candidates. Define

$$X_n = \begin{cases} 1, & \text{if the } n\text{th candidate is the best candidate so far,} \\ 0, & \text{otherwise.} \end{cases}$$

Thus, $X_n = 1$ means that the current candidate is the best among the first n interviewed candidates.

Let $V_m(i)$ denote the maximum probability of selecting the best candidate among all N candidates, given that no one has been hired yet, there are m candidates left to consider, and the current state is

$$i \in \{0, 1\}.$$

Boundary condition. Suppose there are no more candidates left after the current candidate. Then the firm must hire the current candidate. If the current state is $i = 1$, then the current candidate is the best candidate so far and hence is the best overall. If the current state is $i = 0$, then the current candidate is not the best so far and therefore cannot be the best overall. Thus,

$$V_0(1) = 1, \quad V_0(0) = 0.$$

Optimality equations. Suppose there are $m \geq 1$ candidates left to go, and suppose the firm has not hired anyone yet. This means that the firm has already interviewed

$$N - m$$

candidates. Suppose the current state is $i \in \{0, 1\}$.

First consider the case $i = 0$. The current candidate is not the best one so far. If the firm hires this candidate, then the probability that this candidate is the best candidate overall is zero. If the firm does not hire this candidate, then the firm moves to the next candidate. The next candidate will be the best so far with probability

$$\frac{1}{N - m + 1},$$

and the next state will be 0 with probability

$$\frac{N - m}{N - m + 1}.$$

Therefore,

$$V_m(0) = \max \left\{ 0, \frac{1}{N-m+1} V_{m-1}(1) + \frac{N-m}{N-m+1} V_{m-1}(0) \right\}.$$

Since all probabilities and value functions are nonnegative, the second term is always at least zero. Hence,

$$V_m(0) = \frac{1}{N-m+1} V_{m-1}(1) + \frac{N-m}{N-m+1} V_{m-1}(0).$$

The optimal decision in state 0 is to reject the current candidate:

$$d_m(0) = 0.$$

Now consider the case $i = 1$. The current candidate is the best one so far. Given this information, the probability that the current candidate is the best candidate among all N candidates is

$$\frac{N-m}{N}.$$

If the firm hires this candidate, then the process stops and the success probability is $(N-m)/N$.

If the firm does not hire this candidate, then the firm moves to the next candidate. As before, the next candidate will be the best so far with probability

$$\frac{1}{N-m+1},$$

and the next state will be 0 with probability

$$\frac{N-m}{N-m+1}.$$

Therefore,

$$V_m(1) = \max \left\{ \frac{N-m}{N}, \frac{1}{N-m+1} V_{m-1}(1) + \frac{N-m}{N-m+1} V_{m-1}(0) \right\}.$$

Using the expression for $V_m(0)$, we can write this as

$$V_m(1) = \max \left\{ \frac{N-m}{N}, V_m(0) \right\}.$$

Thus, the optimal decision in state 1 is

$$d_m(1) = \begin{cases} 1, & \text{if } \frac{N-m}{N} \geq V_m(0), \\ 0, & \text{if } \frac{N-m}{N} < V_m(0). \end{cases}$$

Example 6.2: Secretary problem with six candidates

Now consider the case

$$N = 6.$$

The boundary values are

$$V_0(1) = 1, \quad V_0(0) = 0.$$

For $m = 1$,

$$V_1(0) = \frac{1}{6} V_0(1) + \frac{5}{6} V_0(0) = \frac{1}{6}.$$

Also,

$$V_1(1) = \max \left\{ \frac{5}{6}, \frac{1}{6} \right\} = \frac{5}{6}.$$

Thus,

$$d_1(1) = 1.$$

For $m = 2$,

$$V_2(0) = \frac{1}{5}V_1(1) + \frac{4}{5}V_1(0) = 0.3.$$

Also,

$$V_2(1) = \max \left\{ \frac{4}{6}, 0.3 \right\} = \frac{2}{3}.$$

Thus,

$$d_2(1) = 1.$$

For $m = 3$,

$$V_3(0) = \frac{1}{4}V_2(1) + \frac{3}{4}V_2(0) = 0.392.$$

Also,

$$V_3(1) = \max \left\{ \frac{3}{6}, 0.392 \right\} = \frac{1}{2}.$$

Thus,

$$d_3(1) = 1.$$

For $m = 4$,

$$V_4(0) = \frac{1}{3}V_3(1) + \frac{2}{3}V_3(0) = 0.428.$$

Also,

$$V_4(1) = \max \left\{ \frac{2}{6}, 0.428 \right\} = 0.428.$$

Thus,

$$d_4(1) = 0.$$

For $m = 5$,

$$V_5(0) = \frac{1}{2}V_4(1) + \frac{1}{2}V_4(0) = 0.428.$$

Also,

$$V_5(1) = \max \left\{ \frac{1}{6}, 0.428 \right\} = 0.428.$$

Thus,

$$d_5(1) = 0.$$

Therefore, the optimal policy for $N = 6$ is

$$[d_5(1), d_4(1), d_3(1), d_2(1), d_1(1)] = [0, 0, 1, 1, 1].$$

That is, when $N = 6$, the firm should reject the first two candidates, and after that hire the first candidate who is the best so far.

6.3 Threshold Structure and the Large Sample Rule

Theorem

For any N , there exists an integer k_N such that the optimal policy for the secretary problem with N candidates is to reject the first k_N candidates and then accept the first candidate who is the best so far.

For the numerical example above,

$$k_6 = 2, \quad \frac{k_6}{6} = \frac{1}{3}.$$

As $N \rightarrow \infty$, it can be shown that

$$\frac{k_N}{N} \rightarrow \frac{1}{e} \approx 0.368.$$

Thus, for large N , the optimal policy is approximately to reject the first 36.8% of candidates and then hire the first candidate after that who is the best so far.

6.4 Parking as a Cost Minimization Problem

Exercise 6.1: Parking lot problem

Imagine driving through a linear parking lot with N parking spaces. The spaces appear one by one from the far end of the parking lot toward the destination.

At each space, the driver observes whether the space is empty or occupied. If the space is empty, denoted by state $i = 0$, then the driver must decide whether to park or continue. Let

$$a = 1$$

denote the action of parking, and let

$$a = 0$$

denote the action of continuing.

If the space is occupied, denoted by state $i = 1$, then the driver cannot park and can only continue:

$$a = 0.$$

Assume that space n is empty with probability p_n . The objective is to minimize the expected distance to the destination. If the driver cannot find a parking space, then the driver incurs a penalty c .

Let $V_m(i)$ denote the minimum expected cost when there are m parking spaces left to inspect and the current state is

$$i \in \{0, 1\}.$$

Here,

$$i = 0$$

means that the current space is empty, and

$$i = 1$$

means that the current space is occupied.

Boundary condition. When there are no more parking spaces left, if the current space is empty, the driver can park immediately and incurs no additional distance cost:

$$V_0(0) = 0.$$

If the current space is occupied and there are no more spaces left, then the driver cannot find a spot and incurs penalty c :

$$V_0(1) = c.$$

Optimality equations. Suppose there are $m \geq 1$ parking spaces left.

First consider the case $i = 0$. The current space is empty. The driver has two choices:

- park now, which gives cost m ;
- continue to the next space.

If the driver continues, then the next space is empty with probability p_m and occupied with probability $1 - p_m$. Therefore, the expected continuation cost is

$$p_m V_{m-1}(0) + (1 - p_m) V_{m-1}(1).$$

Thus,

$$V_m(0) = \min \{m, p_m V_{m-1}(0) + (1 - p_m) V_{m-1}(1)\}.$$

Now consider the case $i = 1$. The current space is occupied, so the driver can only continue. Hence,

$$V_m(1) = p_m V_{m-1}(0) + (1 - p_m) V_{m-1}(1).$$

Therefore, the dynamic programming recursion is

$$V_0(0) = 0, \quad V_0(1) = c,$$

and for $m \geq 1$,

$$\begin{aligned} V_m(0) &= \min \{m, p_m V_{m-1}(0) + (1 - p_m) V_{m-1}(1)\}, \\ V_m(1) &= p_m V_{m-1}(0) + (1 - p_m) V_{m-1}(1). \end{aligned}$$

The optimal action when the current space is empty is

$$a_m^*(0) = \begin{cases} 1, & \text{if } m \leq p_m V_{m-1}(0) + (1 - p_m) V_{m-1}(1), \\ 0, & \text{if } m > p_m V_{m-1}(0) + (1 - p_m) V_{m-1}(1). \end{cases}$$

That is, the driver parks if the cost of parking now is no larger than the expected cost of continuing.

6.5 Comparing the Two Stopping Rules

Both applications reduce a long future to a one dimensional comparison at each stage. In the selection problem, stop when the current success probability exceeds the value of waiting. In the parking problem, stop when the current distance cost is no larger than the expected continuation cost. The state definitions differ, but the Bellman logic is the same.

Lecture 7 Finite Horizon Examples

7.1 Service Control and State Choice

Example 7.1: Managing unscheduled patients

Consider a healthcare clinic that serves both patients with appointments and unscheduled patients. Suppose the clinic offers N appointment slots of equal duration during the day. For example, in an MRI clinic, each slot may correspond to a 20 minute interval.

Assume that all appointment slots are already filled by scheduled patients. However, each scheduled patient actually shows up with probability p , and is a missed appointment with probability $1 - p$. We assume that a scheduled patient who shows up arrives on time.

Unscheduled patients are seen in slots left vacant by scheduled patients who do not arrive, or during overtime. The clinic can control how many unscheduled patients it admits. However, once the clinic accepts a unscheduled patient, it must serve that patient, even if doing so requires overtime.

Each patient, scheduled or unscheduled, produces revenue r . Each unscheduled patient served during overtime costs d , where

$$d > r.$$

The clinic aims to design an optimal acceptance policy for unscheduled patients that maximizes the total expected net revenue over the day.

7.2 Arrivals and State Transitions

Let

$$S_n$$

denote the number of unscheduled patients waiting at the beginning of the n th slot. At the beginning of slot n , the decision maker chooses an action

$$A_n \in \{0, 1, 2, \dots\},$$

where A_n is the maximum number of new unscheduled patients admitted during the n th slot.

Let

$$W_n$$

denote the actual number of unscheduled patients who arrive during the n th slot. Assume that

$$\mathbb{P}(W_n = k) = \alpha_k, \quad k = 0, 1, 2, \dots,$$

for all $1 \leq n \leq N$. The actual number of admitted unscheduled patients during slot n is

$$Y_n = \min(W_n, A_n).$$

For convenience, define the tail probability

$$\beta_a := \mathbb{P}(W_n \geq a) = \sum_{k=a}^{\infty} \alpha_k.$$

Then

$$\mathbb{P}(Y_n = k) = \alpha_k, \quad k = 0, 1, \dots, a - 1,$$

and

$$\mathbb{P}(Y_n = a) = \beta_a.$$

If the scheduled patient in slot n shows up, then the slot is occupied by the scheduled patient, and no waiting unscheduled patient is served during that slot. In this case,

$$S_{n+1} = S_n + Y_n.$$

If the scheduled patient is a missed appointment, then the slot is idle and one of the waiting unscheduled patients is served, provided that $S_n > 0$. Unscheduled patients arriving during slot n cannot be served in the same slot. Hence,

$$S_{n+1} = (S_n - 1)^+ + Y_n.$$

Therefore, conditional on S_n and A_n ,

$$S_{n+1} = \begin{cases} S_n + Y_n, & \text{with probability } p, \\ (S_n - 1)^+ + Y_n, & \text{with probability } 1 - p. \end{cases}$$

Thus,

$$\{(S_n, A_n), n \geq 0\}$$

defines a Markov decision process.

7.3 Clinic Bellman Recursion

Let $V_m(i)$ denote the maximum expected net revenue when there are m more slots left and there are i unscheduled patients waiting at the beginning of the current slot.

When there are no regular slots left, all remaining unscheduled patients must be served in overtime. Each such patient generates revenue r and overtime cost d . Hence,

$$V_0(i) = i(r - d), \quad i \geq 0.$$

Now suppose $1 \leq m \leq N$.

First consider state $i = 0$. There are no unscheduled patients waiting at the beginning of the current slot. If the scheduled patient shows up, which happens with probability p , the clinic earns revenue r . If the scheduled patient does not show up, then the slot remains unused because new unscheduleds arriving in the current slot cannot be served until later. Therefore, the expected immediate revenue is

$$pr.$$

If the clinic chooses action a , then the number of newly admitted unscheduled patients is $Y = \min(W, a)$. Thus,

$$V_m(0) = \max_{a \geq 0} \left\{ pr + \sum_{k=0}^{a-1} \alpha_k V_{m-1}(k) + \beta_a V_{m-1}(a) \right\}.$$

Next consider state $i \geq 1$. There is at least one unscheduled patient waiting at the beginning of the current slot. The clinic earns revenue r during the current slot: either from the scheduled patient if the scheduled patient shows up, or from a waiting unscheduled patient if the scheduled patient is a missed appointment.

If the scheduled patient shows up, then no waiting unscheduled patient is served, and the next state is

$$i + Y.$$

If the scheduled patient is a missed appointment, then one waiting unscheduled patient is served, and the next state is

$$i - 1 + Y.$$

Therefore, for $i \geq 1$,

$$V_m(i) = \max_{a \geq 0} \left\{ r + \sum_{k=0}^{a-1} \alpha_k [pV_{m-1}(i+k) + (1-p)V_{m-1}(i-1+k)] \right. \\ \left. + \beta_a [pV_{m-1}(i+a) + (1-p)V_{m-1}(i-1+a)] \right\}.$$

Together with the terminal condition, these two recursions define the backward induction. At each state, the maximizing value of a is the admission limit for the current slot.

7.4 Sports Control as a Finite Horizon Decision

Example 7.2: When to pull the goalie

In ice hockey, a team can replace its goalie with an extra offensive player at any time during the game. This action is called *pulling the goalie*. Pulling the goalie increases the probability that the team scores, because the team has one additional offensive player on the ice. However, it also increases the probability that the opposing team scores, because the goal is undefended. This strategy is usually used late in a game when a team is behind and wants to tie the game before time expires.

Assume that Team A is behind Team B. We use a discrete time formulation. One time period is chosen to be sufficiently short so that the probability of more than one goal being scored in one time period is negligible.

Let

$$p_A$$

denote the probability that Team A scores one goal in one time period when Team A pulls its goalie, and let

$$p_B$$

denote the probability that Team B scores one goal in one time period when Team A pulls its goalie.

Similarly, let

$$w_A$$

denote the probability that Team A scores one goal in one time period when Team A does not pull its goalie, and let

$$w_B$$

denote the probability that Team B scores one goal in one time period when Team A does not pull its goalie.

We assume

$$p_A > w_A, \quad p_B > w_B.$$

That is, pulling the goalie increases Team A's scoring probability, but it also increases Team B's scoring probability. We also assume that it is possible that neither team scores during a time period, so

$$p_A + p_B < 1, \quad w_A + w_B < 1.$$

State and objective

Let the decision epochs be

$$0, 1, 2, \dots, N,$$

where N is the number of remaining time periods in the game.

The state is the goal differential, defined as

$$\text{Team B's goals} - \text{Team A's goals}.$$

Thus, if the state is i , then Team A is behind by i goals.

Let G be the maximum goal differential at which the coach would still consider pulling the goalie. The state space is

$$S = \{0, 1, 2, \dots, G, G + 1\}.$$

State 0 means that Team A has tied the game or is no longer behind. State $G + 1$ means that Team A is behind by more than G goals. We treat both states as absorbing states:

$$0 \text{ and } G + 1 \text{ are absorbing.}$$

For states $i = 1, 2, \dots, G$, the coach can choose one of two actions:

$$a = 0 : \text{ do not pull the goalie,}$$

$$a = 1 : \text{ pull the goalie.}$$

At the absorbing states, no meaningful action is needed, so we set

$$A_0 = A_{G+1} = \{0\}.$$

The objective is to maximize the probability that Team A ties or wins by the end of the planning horizon. Therefore, rewards are only received at termination.

Let

$$V_m(i)$$

denote the maximum probability that Team A eventually ties or wins, given that there are m decision periods left and the current state is i .

The terminal condition is

$$V_0(0) = 1,$$

and

$$V_0(i) = 0, \quad i = 1, 2, \dots, G, G + 1.$$

For any $m \geq 0$, the absorbing states satisfy

$$V_m(0) = 1, \quad V_m(G + 1) = 0.$$

7.5 Transition Probabilities and Bellman Equation

For $i = 1, 2, \dots, G$, if the coach does not pull the goalie, i.e., $a = 0$, then

$$p_{i,i-1}(0) = w_A,$$

because Team A scores and reduces the goal deficit by one,

$$p_{i,i+1}(0) = w_B,$$

because Team B scores and increases the goal deficit by one, and

$$p_{i,i}(0) = 1 - w_A - w_B,$$

because neither team scores.

If the coach pulls the goalie, i.e., $a = 1$, then

$$p_{i,i-1}(1) = p_A,$$

$$p_{i,i+1}(1) = p_B,$$

and

$$p_{i,i}(1) = 1 - p_A - p_B.$$

The absorbing states satisfy

$$p_{0,0}(0) = 1, \quad p_{G+1,G+1}(0) = 1.$$

Backward recursion

For $i = 1, 2, \dots, G$ and $m \geq 1$, the Bellman equation is

$$V_m(i) = \max_{a \in \{0,1\}} \left\{ \sum_{j \in \{i-1,i,i+1\}} p_{ij}(a) V_{m-1}(j) \right\}.$$

Equivalently,

$$V_m(i) = \max \{ w_A V_{m-1}(i-1) + (1 - w_A - w_B) V_{m-1}(i) + w_B V_{m-1}(i+1), \\ p_A V_{m-1}(i-1) + (1 - p_A - p_B) V_{m-1}(i) + p_B V_{m-1}(i+1) \}.$$

The first term corresponds to not pulling the goalie, and the second term corresponds to pulling the goalie.

The optimal action is

$$d_m(i) = \arg \max_{a \in \{0,1\}} \left\{ \sum_{j \in \{i-1,i,i+1\}} p_{ij}(a) V_{m-1}(j) \right\}.$$

Thus, $d_m(i) = 1$ means that it is optimal to pull the goalie when there are m periods left and Team A is behind by i goals.

7.6 Data Calibration and Policy Interpretation

Example 7.3: Hockey data and implied scoring probabilities

The numerical values below are based on National Hockey League data from the 2013–2020 seasons. When both teams are at full strength, teams score goals at the rate of 2.25 goals per 60 minutes. When a team pulls its goalie, its scoring rate increases to 6.39 goals per 60 minutes. However, the opposing team's scoring rate increases to 19.16 goals per 60 minutes because the net is empty.

If one time period is 5 seconds, then there are 720 time periods in 60 minutes. The corresponding one period scoring probabilities are approximately

$$w_A = w_B = \frac{2.25}{720} = 0.003125,$$

$$p_A = \frac{6.39}{720} \approx 0.008875,$$

and

$$p_B = \frac{19.16}{720} \approx 0.02661.$$

Empirically, teams tend to pull their goalie around 4 minutes before the end of the game when they are behind by three goals, around 2.3 minutes before the end when they are behind by two goals, and around 1.4 minutes before the end when they are behind by one goal. The success rate for pulling the goalie with a one goal deficit is about 14%.

Lecture 8 Discounted MDPs and Bellman Operators

We now consider Markov decision processes over an infinite horizon. Unlike the finite horizon case, the decision process does not stop after a fixed number of periods. Therefore, we introduce a discount factor to compare rewards received at different times.

8.1 Discounted Return and Policy Value

Assume that $\{(S_n, A_n), n \geq 0\}$ is an MDP with state space S and action space A . At time n , the history is

$$H_n = (S_0, A_0, S_1, A_1, \dots, S_{n-1}, A_{n-1}).$$

The Markov property and stationarity mean that

$$\mathbb{P}(S_{n+1} = j \mid S_n = i, A_n = a, H_n) = p_{ij}(a),$$

and

$$\mathbb{E}(G_{n+1} \mid S_n = i, A_n = a, H_n) = g(i, a).$$

Assume the one-period rewards are uniformly bounded in absolute expectation: there exists $B < \infty$ such that

$$\mathbb{E}(|G_{n+1}| \mid S_n = i, A_n = a, H_n) \leq B, \quad i \in S, a \in A(i).$$

Consequently, $|g(i, a)| \leq B$.

Definition

A *policy* describes how actions are chosen over time. We write

$$\pi = [\pi_0, \pi_1, \pi_2, \dots],$$

where π_n specifies how the action at time n is selected. Let Δ denote the set of all policies.

Definition

A policy is called *stationary deterministic* if it uses the same deterministic decision rule at every time period. If $d : S \rightarrow A$ is a decision rule, then the corresponding stationary deterministic policy is

$$\pi = (d, d, d, \dots).$$

Let $\alpha \in (0, 1)$ be the discount factor. A reward received in the future is worth less than a reward received immediately. Specifically, one dollar earned in period $t + n$ is worth α^n dollars earned in period t .

Definition

For any policy π , the *expected total discounted reward* over the infinite horizon, starting in state i and following policy π , is

$$V^\pi(i) = \mathbb{E}_\pi \left[\sum_{n=0}^{\infty} \alpha^n G_{n+1} \mid S_0 = i \right].$$

Equivalently,

$$V^\pi(i) = \mathbb{E}_\pi \left[\sum_{n=0}^{\infty} \alpha^n g(S_n, A_n) \mid S_0 = i \right].$$

The function V^π is called the *value function* of policy π .

The assumption above implies absolute integrability of the discounted sum and

$$|V^\pi(i)| \leq \sum_{n=0}^{\infty} \alpha^n B = \frac{B}{1 - \alpha}, \quad i \in S.$$

Therefore, the expected total discounted reward is well defined.

8.2 Optimal Value Function

Definition

The *optimal value function* is defined by

$$V(i) = \sup_{\pi \in \Delta} V^\pi(i), \quad i \in S.$$

A policy $\pi \in \Delta$ is called *optimal* if

$$V^\pi(i) = V(i), \quad i \in S.$$

When defining the optimal value function, we face three standard questions:

1. Does an optimal policy exist?
2. If an optimal policy exists, is it unique?
3. If an optimal policy exists, how can we characterize it and compute it?

For infinite horizon discounted reward MDPs, an optimal stationary deterministic policy exists under standard assumptions. Therefore, although the set Δ includes history dependent and randomized policies, it is enough to search over stationary deterministic policies.

8.3 Bellman Equation and Policy Recovery

Definition

For an infinite horizon discounted reward MDP, the *Bellman optimality equation* is

$$V(i) = \max_{a \in A(i)} \left\{ g(i, a) + \alpha \sum_{j \in S} p_{ij}(a) V(j) \right\}, \quad i \in S.$$

The term $g(i, a)$ is the immediate expected reward from choosing action a in state i . The term

$$\alpha \sum_{j \in S} p_{ij}(a) V(j)$$

is the discounted expected future value.

Thus, the Bellman equation says that the optimal value in state i equals the best possible sum of immediate reward and discounted expected future value.

Definition

A stationary deterministic policy $\pi = (d, d, d, \dots)$ is called *greedy with respect to V* if, for each state i ,

$$d(i) \in \arg \max_{a \in A(i)} \left\{ g(i, a) + \alpha \sum_{j \in S} p_{ij}(a) V(j) \right\}.$$

If V satisfies the Bellman optimality equation and d is greedy with respect to V , then the stationary deterministic policy

$$\pi = (d, d, d, \dots)$$

is optimal.

Therefore, under standard assumptions, an optimal stationary deterministic policy exists for infinite horizon discounted reward MDPs.

8.4 Bellman Operator and Contraction

The Bellman equation is most useful when it is viewed as a fixed point equation. For a bounded function $u : S \rightarrow \mathbb{R}$, define

$$(Tu)(i) = \max_{a \in A(i)} \left\{ g(i, a) + \alpha \sum_{j \in S} p_{ij}(a) u(j) \right\}.$$

Then the optimal value function satisfies

$$V = TV.$$

The key fact is that T is a contraction in the sup norm. For bounded functions u and v ,

$$\begin{aligned} |(Tu)(i) - (Tv)(i)| &\leq \alpha \max_{a \in A(i)} \sum_j p_{ij}(a) |u(j) - v(j)| \\ &\leq \alpha \|u - v\|_\infty. \end{aligned}$$

Taking the maximum over states gives

$$\|Tu - Tv\|_\infty \leq \alpha \|u - v\|_\infty, \quad 0 < \alpha < 1.$$

Thus one Bellman update reduces the distance to at most an α fraction of its previous value. The contraction mapping theorem therefore gives a unique bounded fixed point V , and repeated Bellman updates converge to it from any bounded starting function.

For a fixed stationary policy π , define

$$(T^\pi u)(i) = g(i, \pi(i)) + \alpha \sum_j p_{ij}(\pi(i)) u(j).$$

The same argument shows

$$\|T^\pi u - T^\pi v\|_\infty \leq \alpha \|u - v\|_\infty,$$

so $V^\pi = T^\pi V^\pi$ also has a unique bounded solution. Once V is known, any rule

$$d(i) \in \arg \max_{a \in A(i)} \left\{ g(i, a) + \alpha \sum_j p_{ij}(a) V(j) \right\}$$

defines an optimal stationary policy.

Here *contraction* is a fixed-point property of an operator; posterior contraction in Lecture 15 is a different statistical notion.

8.5 State Augmentation in a Financial Model

Example 8.1: Stock price dynamics

Consider a stock whose price changes over the state space

$$S = \{0, 1, 2, \dots\}.$$

Let $P = [p_{ij}]$ denote the transition probability matrix of the stock price process. We assume that the stock price evolves independently of our trading decisions.

At any time, we can own either zero or one share of the stock. If we own zero shares, we can either buy one share or do nothing. If we own one share, we can either sell the share or do nothing. When we buy a share, our cash reserve decreases by the current stock price. When we sell the share, our cash reserve increases by the current stock price. We allow borrowing, so the cash reserve is not constrained to be nonnegative.

The objective is to find a trading policy that maximizes the expected total discounted cash flow over the infinite horizon.

Let Y_n be the stock price on day n , and let Z_n be the number of shares owned on day n before any trade is made. The state of the system is

$$S_n = (Y_n, Z_n),$$

where $Y_n \in S$ and $Z_n \in \{0, 1\}$.

There are three possible actions:

$a = 0$ do nothing,

$a = 1$ buy one share,

$a = 2$ sell one share.

The feasible actions depend on the number of shares currently owned. If $Z_n = 0$, then the feasible actions are

$$A(i, 0) = \{0, 1\}.$$

If $Z_n = 1$, then the feasible actions are

$$A(i, 1) = \{0, 2\}.$$

The immediate rewards are

$$g((i, 0), 0) = 0, \quad g((i, 0), 1) = -i,$$

and

$$g((i, 1), 0) = 0, \quad g((i, 1), 2) = i.$$

Thus, buying generates an immediate cash flow of $-i$, selling generates an immediate cash flow of i , and doing nothing generates an immediate cash flow of 0.

Since the trading decision does not affect the evolution of the stock price, the next stock price follows the same transition matrix P . If we buy, the next share ownership state is 1. If we sell, the next share ownership state is 0. If we do nothing, the next share ownership state remains unchanged.

Let $V(i, k)$ be the optimal expected total discounted reward starting from state (i, k) , where i is the current stock price and $k \in \{0, 1\}$ is the current number of shares owned. Then the Bellman optimality equations are, for all $i \in S$,

$$V(i, 0) = \max \left\{ \alpha \sum_{j \in S} p_{ij} V(j, 0), -i + \alpha \sum_{j \in S} p_{ij} V(j, 1) \right\},$$

and

$$V(i, 1) = \max \left\{ \alpha \sum_{j \in S} p_{ij} V(j, 1), i + \alpha \sum_{j \in S} p_{ij} V(j, 0) \right\}.$$

The first equation corresponds to the case where we currently own no share. The first term represents doing nothing, and the second term represents buying one share. The second equation corresponds to the case where we currently own one share. The first term represents holding the share, and the second term represents selling it.

An optimal policy can be obtained by comparing the two terms in each Bellman equation. In state $(i, 0)$, it is optimal to buy if

$$-i + \alpha \sum_{j \in S} p_{ij} V(j, 1) \geq \alpha \sum_{j \in S} p_{ij} V(j, 0).$$

In state $(i, 1)$, it is optimal to sell if

$$i + \alpha \sum_{j \in S} p_{ij} V(j, 0) \geq \alpha \sum_{j \in S} p_{ij} V(j, 1).$$

Lecture 9 Computing Discounted Policies

In the infinite horizon discounted reward model, the optimal value function satisfies the Bellman optimality equation

$$V(i) = \max_{a \in A(i)} \left\{ g(i, a) + \alpha \sum_{j \in S} p_{ij}(a) V(j) \right\}, \quad i \in S.$$

This equation is a functional equation because the unknown object is the function $V : S \rightarrow \mathbb{R}$. We now introduce several standard methods for solving this equation:

1. value iteration;
2. policy iteration;
3. linear programming.

9.1 Value Iteration

Value iteration approximates the optimal value function by repeatedly applying the Bellman operator.

Let V_0 be an arbitrary bounded function on S . A common choice is

$$V_0(i) = 0, \quad i \in S.$$

For $n \geq 1$, define

$$V_n(i) = \max_{a \in A(i)} \left\{ g(i, a) + \alpha \sum_{j \in S} p_{ij}(a) V_{n-1}(j) \right\}, \quad i \in S.$$

Equivalently,

$$V_n = TV_{n-1},$$

where T is the Bellman optimality operator.

From the finite horizon model, $V_n(i)$ can be interpreted as the optimal expected total discounted reward over periods

$$0, 1, \dots, n-1$$

starting from state i , with zero terminal reward.

Theorem

Let V be the optimal value function for the infinite horizon discounted reward problem. Then

$$\lim_{n \rightarrow \infty} V_n(i) = V(i), \quad i \in S.$$

Moreover, the convergence is uniform over S .

In practice, value iteration must be stopped after a finite number of iterations. Suppose we stop after n iterations and obtain V_n . If

$$\|V_n - V\|_\infty \leq \epsilon,$$

then V_n is an ϵ -accurate approximation to the optimal value function.

A natural way to construct a stationary deterministic Markovian policy from V_n is to choose a policy π induced by V_n , that is,

$$\pi(i) \in \arg \max_{a \in A(i)} \left\{ g(i, a) + \alpha \sum_{j \in S} p_{ij}(a) V_n(j) \right\}, \quad i \in S.$$

Theorem

Suppose $\|V_n - V\|_\infty \leq \epsilon$, and let π be a policy induced by V_n . Then the value function V^π of policy π satisfies

$$V(i) - \frac{2\epsilon}{1-\alpha} \leq V^\pi(i) \leq V(i), \quad i \in S.$$

This theorem shows that if V_n is close to the optimal value function, then the policy induced by V_n is also close to optimal.

9.2 Policy Iteration

Policy iteration improves policies directly. It alternates between evaluating the current policy and improving the policy using the current value function.

1. **Initialization.** Let π_0 be an arbitrary stationary deterministic Markovian policy. Set $n = 0$.
2. **Policy evaluation.** Compute V^{π_n} , the value function of policy π_n . That is, V^{π_n} satisfies

$$V^{\pi_n}(i) = g(i, \pi_n(i)) + \alpha \sum_{j \in S} p_{ij}(\pi_n(i)) V^{\pi_n}(j), \quad i \in S.$$

3. **Policy improvement.** Choose a policy π_{n+1} induced by V^{π_n} :

$$\pi_{n+1}(i) \in \arg \max_{a \in A(i)} \left\{ g(i, a) + \alpha \sum_{j \in S} p_{ij}(a) V^{\pi_n}(j) \right\}, \quad i \in S.$$

4. **Stopping rule.** Compute $V^{\pi_{n+1}}$, the value function of policy π_{n+1} . If

$$V^{\pi_{n+1}} = V^{\pi_n},$$

stop. Otherwise, set $n = n + 1$ and return to the policy improvement step.

Theorem

For policy iteration,

$$V^{\pi_{n+1}} \geq V^{\pi_n}.$$

If

$$V^{\pi_{n+1}} = V^{\pi_n},$$

then $V^{\pi_{n+1}}$ is the optimal value function and π_{n+1} is an optimal policy.

Now suppose both the state space and the action space are finite. Let

$$|A| = n_A, \quad |S| = n_S.$$

A stationary deterministic Markovian policy is a function $\pi : S \rightarrow A$, so there are at most $n_A^{n_S}$ such policies. Hence, policy iteration terminates in a finite number of steps.

Policy iteration is usually more involved than value iteration because policy evaluation requires solving simultaneous linear equations; the two methods can also be combined.

Example 9.1: One policy improvement step for the stock model

Return to the stock model from Lecture 8 and restrict prices to $S = \{0, 1, 2, 3\}$ with

$$P = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0.3 & 0 & 0.7 & 0 \\ 0 & 0.7 & 0 & 0.3 \\ 0 & 0 & 1 & 0 \end{pmatrix}, \quad \alpha = 0.9.$$

The augmented state is (i, k) , where i is price and $k \in \{0, 1\}$ records share ownership. Start with a policy π_0 that buys at $(0, 0)$, sells at $(3, 1)$, and otherwise does nothing.

Policy evaluation does not require writing eight Bellman equations separately. With the eight augmented states in a fixed order, write the reward vector as g^{π_0} and the induced transition matrix as P^{π_0} . Then

$$V^{\pi_0} = g^{\pi_0} + \alpha P^{\pi_0} V^{\pi_0}, \quad (I - \alpha P^{\pi_0}) V^{\pi_0} = g^{\pi_0}.$$

Solving this linear system gives, in the order $(0, 0), (1, 0), (2, 0), (3, 0), (0, 1), (1, 1), (2, 1), (3, 1)$,

$$V^{\pi_0} \approx (1.5568, 0.8836, 0.7354, 0.6618, 1.5568, 1.7297, 2.0784, 3.6618)^\top.$$

The improvement step compares the feasible one step look ahead values

$$g(s, a) + \alpha \sum_{s'} p(s' | s, a) V^{\pi_0}(s').$$

It keeps “buy” only at $(0, 0)$ and changes the owned share states so that selling is preferred at prices 1, 2, and 3. The point of the example is the algorithmic pattern: solve one linear system for the current policy, then make local action comparisons. Repeating these two steps gives policy iteration.

9.3 Primal Linear Program

The Bellman equation also gives a linear program when the state and action sets are finite. A function u is an upper bound on the optimal value whenever

$$u(i) \geq g(i, a) + \alpha \sum_{j \in S} p_{ij}(a) u(j), \quad i \in S, \ a \in A(i).$$

Thus, for any strictly positive weights $b(i)$ with $\sum_i b(i) = 1$, solve

$$\begin{aligned} & \text{minimize} && \sum_i b(i) u(i) \\ & \text{subject to} && u(i) - \alpha \sum_j p_{ij}(a) u(j) \geq g(i, a), \quad i \in S, \ a \in A(i). \end{aligned}$$

The optimal solution is V . At each state, at least one Bellman inequality is tight; choosing a tight action recovers an optimal policy. The weights $b(i)$ select among upper bounds in the objective but do not change the Bellman constraints.

9.4 Occupation Measures and the Dual Program

Use $\eta(i, a) \geq 0$ for the dual variable associated with state i and action a , reserving π for policies. The dual program is

$$\begin{aligned} & \text{maximize} && \sum_{i,a} g(i, a) \eta(i, a) \\ & \text{subject to} && \sum_a \eta(j, a) - \alpha \sum_{i,a} p_{ij}(a) \eta(i, a) = b(j), \quad j \in S. \end{aligned}$$

For a stationary policy and initial distribution b ,

$$\eta(i, a) = \sum_{n=0}^{\infty} \alpha^n \Pr(S_n = i, A_n = a)$$

is the discounted expected number of visits to the state and action pair. Consequently the dual objective is the expected total discounted reward. Complementary slackness links the two views: positive occupation mass is placed only on actions whose Bellman inequality is tight. The primal therefore describes optimal value through local inequalities, while the dual describes the same solution through discounted state and action flow.

Lecture 10 Continuous Time Control and Long Run Criteria

10.1 Decisions at Continuous Time Events

So far, we have mainly considered discrete time decision models. In many applications, however, the underlying system evolves in continuous time. Continuous time models can often be reduced to discrete time MDPs when decisions are made only at discrete event epochs.

Consider a continuous time Markov chain

$$\{X(t), t \geq 0\}$$

with state space

$$S = \{0, 1, 2, \dots\}.$$

Suppose the current state is i . There are several event types, $k = 1, 2, \dots$. Conditional on state i , assume their event clocks are independent, with clock k exponential with rate $q_{i,k}$. When event k occurs, the decision maker observes it and chooses an action a . The system then jumps to state j with probability

$$p_{i,k,j}(a).$$

Let $\delta > 0$ be the continuous time discount rate. A dollar earned at time t is worth

$$e^{-\delta t}$$

dollars at time 0. Therefore, future rewards are discounted continuously over time.

For each state i , assume the total event rate is finite and define

$$q_i = \sum_k q_{i,k} < \infty.$$

The time until the next event is then exponential with rate q_i .

10.2 Discounting Between Events

We need two basic calculations for exponential event times.

First, suppose a lump sum reward r is earned at the event time T , where

$$T \sim \exp(q).$$

The expected discounted reward at time 0 is

$$\mathbb{E}(re^{-\delta T}) = r\mathbb{E}(e^{-\delta T}).$$

Since $T \sim \exp(q)$,

$$\mathbb{E}(e^{-\delta T}) = \int_0^\infty e^{-\delta t} q e^{-qt} dt = q \int_0^\infty e^{-(q+\delta)t} dt = \frac{q}{q+\delta}.$$

Hence,

$$\mathbb{E}(re^{-\delta T}) = \frac{q}{q+\delta} r.$$

Second, suppose reward is earned continuously at a constant rate r over the time interval $[0, T]$. The expected total discounted reward at time 0 is

$$\mathbb{E}\left(\int_0^T r e^{-\delta t} dt\right).$$

Using the survival probability $P(T > t) = e^{-qt}$, we have

$$\mathbb{E} \left(\int_0^T r e^{-\delta t} dt \right) = \int_0^\infty r e^{-\delta t} P(T > t) dt = \int_0^\infty r e^{-\delta t} e^{-qt} dt.$$

Therefore,

$$\mathbb{E} \left(\int_0^T r e^{-\delta t} dt \right) = r \int_0^\infty e^{-(q+\delta)t} dt = \frac{r}{q+\delta}.$$

Now suppose the system earns rewards at rate $u(i)$ while it is in state i . In addition, when event k occurs and action a is chosen, the system jumps to state j instantly and earns an additional expected lump sum reward

$$u'((i, k, j), a).$$

Let $v(i)$ denote the expected total discounted reward over $[0, \infty)$ when the system is initially in state i . We can derive the optimality equation by conditioning on the first event.

While the system remains in state i , it earns reward at rate $u(i)$. Since the time until the next event is exponential with rate q_i , the expected discounted reward earned before the first event is

$$\frac{u(i)}{q_i + \delta}.$$

Next, event k occurs first with probability

$$\frac{q_{i,k}}{q_i}.$$

The discount factor associated with the first event time contributes the factor

$$\frac{q_i}{q_i + \delta}.$$

Multiplying these two terms gives

$$\frac{q_{i,k}}{q_i + \delta}.$$

Therefore, after the first event, the decision maker chooses an action a to maximize the expected lump sum reward plus the continuation value. The optimality equation is

$$v(i) = \frac{u(i)}{q_i + \delta} + \sum_k \frac{q_{i,k}}{q_i + \delta} \max_{a \in A} \left\{ \sum_{j \in S} p_{i,k,j}(a) u'((i, k, j), a) + \sum_{j \in S} p_{i,k,j}(a) v(j) \right\}, \quad i \in S.$$

Equivalently,

$$v(i) = \frac{u(i)}{q_i + \delta} + \sum_k \frac{q_{i,k}}{q_i + \delta} \max_{a \in A} \left\{ \sum_{j \in S} p_{i,k,j}(a) [u'((i, k, j), a) + v(j)] \right\}, \quad i \in S.$$

This equation converts the continuous time decision problem into a discrete event dynamic programming equation. The decision epochs are the event times of the CTMC, and the effective discounting is incorporated through the factors

$$\frac{1}{q_i + \delta} \quad \text{and} \quad \frac{q_{i,k}}{q_i + \delta}.$$

10.3 The Average Reward Criterion

In the discounted reward model, future rewards are multiplied by a discount factor. Another important infinite horizon criterion is the *average reward* criterion. Instead of discounting future rewards, the average reward model evaluates the long run reward per period.

Consider a basic MDP

$$\{(S_n, A_n), n \geq 0\},$$

with state space S , action space A , transition probability matrices

$$P(a) = [p_{i,j}(a)], \quad i, j \in S, \quad a \in A,$$

and bounded expected one period rewards

$$g(i, a), \quad i \in S, \quad a \in A.$$

Let Δ denote the set of all admissible policies.

Definition

For a policy $\pi \in \Delta$, define

$$\rho^\pi(i) = \lim_{n \rightarrow \infty} \frac{1}{n+1} \mathbb{E}_\pi \left(\sum_{k=0}^n g(S_k, A_k) \mid S_0 = i \right), \quad i \in S,$$

provided that the limit exists.

We call $\rho^\pi(i)$ the **long run average reward** starting from state i under policy π .

The optimal average reward starting from state i is defined by

$$\rho^*(i) = \sup\{\rho^\pi(i) : \pi \in \Delta\}.$$

Definition

A policy $\pi \in \Delta$ is **average reward optimal** if

$$\rho^\pi(i) = \rho^*(i), \quad i \in S.$$

As in the discounted reward model, we face three central questions:

1. Does an average reward optimal policy exist?
2. If an average reward optimal policy exists, is it unique?
3. If an average reward optimal policy exists, how can we characterize it and compute it?

These questions are more delicate under the average reward criterion than under the discounted reward criterion. In the discounted reward model, bounded rewards and a discount factor $\alpha \in (0, 1)$ give strong contraction properties. In the average reward model, there is no discount factor, so convergence and optimality may depend more heavily on the structure of the Markov chain induced by the policy.

Average reward problems need assumptions that discounted problems often avoid. For a fixed stationary policy, the induced Markov chain should have stable long run behavior. In finite communicating or unichain models this is usually manageable; in general state spaces, the limit defining $\rho^\pi(i)$ can fail to exist or can depend on the initial state.

Example 10.1: Why the average reward limit can fail

Let the state move deterministically as $0, 1, 2, \dots$ and let the reward along this path alternate between increasingly long blocks of zeros and ones. The running average can then oscillate rather than converge. For example, choosing block lengths so that the limsup is $1/2$ and the liminf is $1/3$ gives a bounded reward process for which $\rho^\pi(0)$ is undefined.

A second issue is attainment. Even when every policy has a well defined average reward, a sequence of stationary policies can approach a supremum that no stationary policy attains. This is why later Bellman equations are stated under finite communicating type assumptions rather than for arbitrary MDPs. The warning is conceptual: before using an average reward algorithm, verify that the model has enough recurrence and communication for one long run rate to be meaningful.

10.4 From Discounted to Average Reward

Discounted and average reward criteria emphasize different horizons. With discount factor $\alpha < 1$, the Bellman operator is a contraction and future rewards receive geometrically smaller weights. Average reward treats periods symmetrically and depends on the recurrent behavior of the controlled chain.

For a fixed policy π , let

$$\phi_\alpha^\pi(i) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \alpha^k g(S_k, A_k) \mid S_0 = i \right].$$

If the long run average reward $\rho^\pi(i)$ exists, an Abelian limit gives

$$\lim_{\alpha \uparrow 1} (1 - \alpha) \phi_\alpha^\pi(i) = \rho^\pi(i).$$

Thus a lightly discounted criterion approximates the long run rate after the normalization $1 - \alpha$. The converse requires care, and optimization cannot always be interchanged with the limit. Lecture 11 develops the average reward Bellman equation under assumptions that make this passage well behaved.

Lecture 11 Average Reward Bellman Equations and Algorithms

11.1 Average Reward and Relative Value

Consider a finite MDP with states $i \in S$, feasible actions $a \in A(i)$, transition probabilities $p_{ij}(a)$, and expected one period reward $g(i, a)$. For a policy π , define its average reward from state i by

$$\rho^\pi(i) = \lim_{N \rightarrow \infty} \frac{1}{N} \mathbb{E}_i^\pi \left[\sum_{t=0}^{N-1} g(S_t, A_t) \right],$$

when the limit exists. In a communicating finite model, the optimal value is often the same for every initial state; we denote it by ρ^* .

Average reward alone does not distinguish states that differ during the first few periods. The *relative value*, or *bias*, $h(i)$ records that transient advantage. Informally, $h(i) - h(j)$ is the extra reward obtained by starting in i rather than j before the common long run rate dominates. Because only differences matter, we choose one reference state s_0 and set $h(s_0) = 0$.

The assumption that one common rate applies to every initial state is not automatic. A convenient sufficient setting is a finite communicating model: under suitable stationary policies the relevant states can reach one another. A slightly broader condition, often called a unichain condition, permits transient states but requires one recurrent class under each stationary policy. These conditions keep the long run criterion from splitting into unrelated state dependent problems. When they fail, one may need a vector of average rewards rather than the single scalar ρ used here.

The bias also has a useful finite horizon interpretation. If a policy has average reward ρ and bias h , then for a long horizon N ,

$$\mathbb{E}_i^\pi \left[\sum_{t=0}^{N-1} g(S_t, A_t) \right] \approx N\rho + h(i) - \mathbb{E}_i^\pi[h(S_N)].$$

The leading term is the common long run rate. The bounded correction explains why two starting states can matter for a while even though their average reward is the same.

11.2 The Average Reward Bellman Equation

The central equation is

$$\rho + h(i) = \max_{a \in A(i)} \left\{ g(i, a) + \sum_{j \in S} p_{ij}(a) h(j) \right\}, \quad i \in S. \quad (1)$$

The scalar ρ measures reward per period. The function h compares the next state with the current state after that common rate has been removed.

Theorem

Suppose a scalar ρ and a bounded function h satisfy (1). Choose, in each state i , an action $a^*(i)$ that attains the maximum. Then the stationary policy $\pi^*(i) = a^*(i)$ has average reward ρ , and no policy has a larger average reward under the standard finite communicating assumptions.

To see the logic, rewrite the equality for the selected action as

$$g(i, a^*(i)) - \rho = h(i) - \sum_j p_{ij}(a^*(i)) h(j).$$

Along a trajectory, the right side telescopes in expectation. After division by the horizon, the bounded endpoint terms vanish, leaving the rate ρ . For any other action, the Bellman inequality gives an upper bound by the same argument.

A second derivation starts from the N period value function. If, for large N , one expects

$$V_N(i) \approx N\rho + h(i),$$

then the finite horizon recursion

$$V_{N+1}(i) = \max_a \left\{ g(i, a) + \sum_j p_{ij}(a) V_N(j) \right\}$$

gives, after cancelling $N\rho$, exactly

$$\rho + h(i) = \max_a \left\{ g(i, a) + \sum_j p_{ij}(a) h(j) \right\}.$$

This calculation is informal, but it shows where the extra scalar comes from: one more period contributes approximately one more unit of average reward.

11.3 The Discounted Limit

Let $V_\alpha(i)$ be the optimal discounted value with discount factor $\alpha \in (0, 1)$:

$$V_\alpha(i) = \max_{a \in A(i)} \left\{ g(i, a) + \alpha \sum_j p_{ij}(a) V_\alpha(j) \right\}.$$

Fix s_0 and separate level from differences:

$$\rho_\alpha = (1 - \alpha)V_\alpha(s_0), \quad h_\alpha(i) = V_\alpha(i) - V_\alpha(s_0).$$

If $\rho_\alpha \rightarrow \rho$ and $h_\alpha(i) \rightarrow h(i)$ as $\alpha \uparrow 1$, then passing to the limit gives (1). This explains both the average rate and the normalization of the bias. It also explains why the average reward problem is harder: the discounted Bellman operator is a contraction, while the average reward equation is unchanged when a constant is added to h .

11.4 Solving a Finite Model

Example 11.1: A four state calculation

Let $S = \{1, 2, 3, 4\}$ and $A(i) = \{1, 2\}$. Suppose the two transition matrices are

$$P(1) = \begin{pmatrix} .1 & .2 & .3 & .4 \\ 0 & .5 & 0 & .5 \\ 0 & .1 & 0 & .9 \\ .2 & .4 & 0 & .4 \end{pmatrix}, \quad P(2) = \begin{pmatrix} .1 & .2 & .3 & .4 \\ 0 & .3 & 0 & .7 \\ .5 & 0 & .5 & 0 \\ .3 & 0 & .6 & .1 \end{pmatrix},$$

and the reward matrix $G = [g(i, a)]$ is

$$G = \begin{pmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \\ 7 & 8 \end{pmatrix}.$$

Set $h(1) = 0$. For state 2, the Bellman equation is

$$\rho + h(2) = \max\{3 + .5h(2) + .5h(4), 4 + .3h(2) + .7h(4)\}.$$

Writing the analogous equations for states 1, 3, 4 gives four equations in $\rho, h(2), h(3), h(4)$ once an action is selected in each state. A practical solution checks the 2^4 stationary policies, solves the corresponding linear system, and verifies that the chosen action attains the maximum in every state.

This calculation is useful because it separates two tasks: solve linear equations for a proposed policy, then check Bellman inequalities to improve or validate the policy.

For a proposed deterministic policy d , policy evaluation solves

$$\rho_d + h_d(i) = g(i, d(i)) + \sum_j p_{ij}(d(i))h_d(j), \quad h_d(s_0) = 0.$$

Policy improvement then replaces $d(i)$ by an action maximizing $g(i, a) + \sum_j p_{ij}(a)h_d(j)$. If no action changes, the Bellman equation is satisfied. This is the average reward analogue of policy iteration from Lecture 9.

11.5 Relative Value Iteration

Choose a reference state s_0 and initialize $h_0(i) = 0$. Define

$$(Th)(i) = \max_{a \in A(i)} \left\{ g(i, a) + \sum_j p_{ij}(a)h(j) \right\}.$$

Relative value iteration repeats

$$h_{k+1}(i) = (Th_k)(i) - (Th_k)(s_0).$$

The subtraction fixes the arbitrary additive constant. A useful stopping measure is the span

Normalization does not change action comparisons because every candidate action adds the same constant through $\sum_j p_{ij}(a) = 1$. It only prevents the iterates from drifting by an irrelevant offset. The natural error measure is therefore not $\max_i |h(i)|$, which changes under a constant shift, but the span of a vector.

$$\text{sp}(h_{k+1} - h_k) = \max_i \{h_{k+1}(i) - h_k(i)\} - \min_i \{h_{k+1}(i) - h_k(i)\}.$$

When this span is small, $(Th_k)(s_0)$ estimates ρ^* , and a greedy action with respect to h_k gives an approximate policy.

11.6 Linear Programs and Occupation Measures

A primal linear program is

$$\begin{aligned} & \text{minimize} && \rho \\ & \text{subject to} && \rho + h(i) \geq g(i, a) + \sum_j p_{ij}(a)h(j), \quad i \in S, \quad a \in A(i). \end{aligned}$$

Its dual uses occupation fractions $\eta(i, a)$:

$$\begin{aligned} & \text{maximize} && \sum_{i,a} g(i, a)\eta(i, a) \\ & \text{subject to} && \sum_a \eta(j, a) = \sum_{i,a} p_{ij}(a)\eta(i, a), \quad j \in S, \\ & && \sum_{i,a} \eta(i, a) = 1, \quad \eta(i, a) \geq 0. \end{aligned}$$

The balance equation says that long run flow into each state equals long run flow out. If $d(i) = \sum_a \eta(i, a) > 0$, the dual solution induces the stationary randomized policy

$$\pi(a \mid i) = \frac{\eta(i, a)}{d(i)}.$$

Thus Bellman equations, relative value iteration, and linear programming are three views of the same average reward problem: local optimality, repeated computation, and long run flow.

Complementary slackness connects the two linear programs. Positive occupation mass can occur only on actions whose primal Bellman inequality is tight. In a small model, this provides a useful check: the action selected from h should agree with the action carrying mass in η . Disagreement usually signals an incorrect normalization, a transcription error, or a nonunique optimal policy.

Lecture 12 Learning Value Functions from Experience

12.1 From a Known Model to Sampled Experience

In a known MDP, the transition probabilities $p(s' | s, a)$ and expected reward $g(s, a)$ appear directly in a Bellman equation. Reinforcement learning begins when these quantities are not available and the learner instead observes transitions

$$(S_t, A_t, G_{t+1}, S_{t+1}),$$

where G_{t+1} is the reward observed after action A_t . We reserve r_t for regret later in the notes.

For a fixed policy π , the discounted value is

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \alpha^k G_{t+k+1} \mid S_t = s \right], \quad 0 < \alpha < 1.$$

Learning can target the model, the value function, or the policy itself.

The distinction between a random reward and its mean is important. The observed quantity is G_{t+1} , while

$$g(s, a) = \mathbb{E}[G_{t+1} \mid S_t = s, A_t = a]$$

is the expected one period reward used in a Bellman equation. A single transition is noisy evidence about both g and the next state distribution. Repeated visits are what turn that evidence into a stable estimate.

Example 12.1: A gridworld trajectory

An agent moves on a 5×5 grid. The goal gives reward R , a trap gives reward $-C$, and every other move gives reward -1 . The actions are up, down, left, and right, but motion is noisy. If the transition law is known, dynamic programming finds a route. If it is unknown, the agent must learn from paths such as

$$(S_0, A_0, G_1, S_1, A_1, G_2, S_2, \dots).$$

The same data can estimate transition probabilities or update values directly.

12.2 Model Estimation and Direct Value Learning

A model based method counts transitions. Let $N_t(s, a)$ be the number of visits to (s, a) by time t and $N_t(s, a, s')$ the number followed by state s' . Then

$$\hat{p}_t(s' | s, a) = \frac{N_t(s, a, s')}{N_t(s, a)}$$

when the denominator is positive. Sample averages estimate $g(s, a)$. The estimated MDP can then be solved by value iteration or policy iteration.

A direct, or model free, method avoids this intermediate model and updates a value estimate from each observed transition. The advantage is that it need not estimate probabilities that are irrelevant to the final policy. The cost is that values may require many visits, and the data still must cover useful state and action pairs. In either approach, exploration is a statistical design problem: the learner controls which observations become available.

The two routes use data differently. A learned transition model can be reused for many objectives, discount factors, or policy constraints. Direct value learning can be more economical when only one objective matters. Model error also propagates through planning: a small probability error can matter

greatly if it changes access to a valuable state. Direct methods avoid an explicit probability estimate, but they do not avoid approximation error or poor data coverage.

A useful classroom check is to ask what changes after one transition. In a model based method, counts and reward averages change first, and a planner is run afterward. In a direct method, the value estimate itself changes. The observed tuple is identical; only the summary retained by the algorithm differs.

12.3 Monte Carlo Policy Evaluation

Suppose episodes end at a terminal state and all episodes start from the same state s . For episode k , let

$$U_k = \sum_{t=0}^{\tau_k-1} \alpha^t G_{t+1}^{(k)}$$

be the realized return. The Monte Carlo estimator is

$$\hat{V}_K^\pi(s) = \frac{1}{K} \sum_{k=1}^K U_k.$$

If the episodes are independent, follow π , and have integrable returns, the law of large numbers gives $\hat{V}_K^\pi(s) \rightarrow V^\pi(s)$. The estimator is simple and uses no transition model. Its limitation is that an update must wait until the episode ends, and a long return can have substantial variance.

For every visit evaluation, one also forms a return from each time that state s is visited. This reuses an episode more efficiently but introduces dependence among returns from the same path. The averaging principle remains the same.

The sample mean can be updated without storing all returns:

$$\hat{V}_K^\pi(s) = \hat{V}_{K-1}^\pi(s) + \frac{1}{K} \{U_K - \hat{V}_{K-1}^\pi(s)\}.$$

This is the same recursive average used for bandit means. It also reveals the learning rate $1/K$: early episodes move the estimate substantially, while later episodes make smaller corrections.

Monte Carlo evaluation has no bootstrapping bias because the target is the realized return. Its variance can still be large, especially when rewards are delayed. This bias and variance comparison motivates temporal difference methods rather than proving that one method is always preferable.

12.4 Temporal Difference Learning

Temporal difference learning updates after one transition. With current estimate v_t , define

$$\delta_t = G_{t+1} + \alpha v_t(S_{t+1}) - v_t(S_t).$$

The update is

$$v_{t+1}(S_t) = v_t(S_t) + \eta_t \delta_t,$$

with all other states unchanged. The next state estimate supplies a provisional continuation value, a step often called bootstrapping. Compared with a full Monte Carlo return, the target is available sooner and often has lower variance, but it can be biased while v_t is inaccurate.

Example 12.2: One temporal difference update

Let $\alpha = 0.9$, $v_t(s) = 3$, $v_t(s') = 5$, and suppose the transition from s to s' gives $G_{t+1} = -1$. Then

$$\delta_t = -1 + 0.9(5) - 3 = 0.5.$$

With $\eta_t = 0.2$, the new estimate is $3 + 0.2(0.5) = 3.1$. The observed one step target was slightly larger than the previous estimate.

Common learning rates satisfy

$$\eta_t > 0, \quad \sum_t \eta_t = \infty, \quad \sum_t \eta_t^2 < \infty.$$

The first sum allows persistent correction; the second limits accumulated noise.

Under a fixed policy, the expected temporal difference is zero at the true value function because

$$V^\pi(s) = g_\pi(s) + \alpha \sum_{s'} p_\pi(s' | s) V^\pi(s').$$

Thus TD learning can be viewed as a stochastic approximation to the Bellman fixed point. The update replaces the unavailable conditional expectation by one sampled reward and one sampled next state.

12.5 Q Learning and Exploration

For control, learn the optimal action value $Q^*(s, a)$ rather than only a state value. With estimate q_t , Q learning uses

$$q_{t+1}(S_t, A_t) = q_t(S_t, A_t) + \eta_t [G_{t+1} + \alpha \max_{a'} q_t(S_{t+1}, a') - q_t(S_t, A_t)].$$

A greedy policy then chooses an action in $\arg \max_a q_t(s, a)$. The action used to collect data may differ from this greedy action, so Q learning is an off policy method. For a finite MDP with bounded rewards, suitable learning rates, and infinitely many visits to every relevant state and action pair, the tabular estimate converges to Q^* .

The visit condition is not automatic. Always taking the current greedy action can leave alternatives untested, while random action selection wastes reward. The next lecture isolates this exploration problem in the simpler bandit model.

Example 12.3: One Q learning update

Suppose $q_t(s, a) = 2.0$, the observed reward is $G_{t+1} = 1$, the largest next state value is $\max_{a'} q_t(s', a') = 3$, the discount factor is 0.9, and the learning rate is 0.1. The target is $1 + 0.9(3) = 3.7$, so

$$q_{t+1}(s, a) = 2.0 + 0.1(3.7 - 2.0) = 2.17.$$

Only the visited pair changes. Future visits gradually average many such noisy targets.

It is useful to distinguish the behavior policy, which generates data, from the target policy represented by the maximum in the update. Q learning can explore with one rule while learning the greedy policy. That flexibility is valuable, but it makes coverage an explicit responsibility of the behavior rule.

Lecture 13 Bandits and Directed Exploration

13.1 The Bandit Model and Regret

A stochastic bandit has arms $a \in \{1, \dots, K\}$. Pulling arm a produces a reward Y_t with unknown mean μ_a . Rewards are independent across rounds conditional on the selected arms. Let

$$a^* \in \arg \max_a \mu_a, \quad \mu^* = \mu_{a^*}.$$

If arm a_t is selected at round t , define

$$r_t = \mu^* - \mu_{a_t}, \quad R_T = \sum_{t=1}^T r_t.$$

The first is instantaneous regret and the second is cumulative regret. If $N_a(T)$ counts pulls of arm a and $\Delta_a = \mu^* - \mu_a$, then

$$R_T = \sum_{a \neq a^*} \Delta_a N_a(T).$$

This identity turns regret analysis into a question about how often a learning rule selects suboptimal arms.

Regret is often random because the selected arms depend on random rewards. A common theorem bounds expected regret $\mathbb{E}[R_T]$ or gives a statement that holds with high probability. The expression above uses the unknown means and is sometimes called pseudo regret; realized regret would compare the actual reward sequence with a coupled oracle sequence. For the stochastic models used here, the mean based criterion is the cleaner object and directly measures the cost of learning which arm is best.

The gap Δ_a determines difficulty. A large gap is easy to detect, while two nearly tied arms require many observations. A bound that depends on the gaps is informative for a fixed problem; a gap free bound describes worst case behavior when the gaps may be arbitrarily small.

13.2 Empirical Means and Random Exploration

After $N_a(t)$ observations from arm a , its empirical mean is

$$\hat{\mu}_a(t) = \frac{1}{N_a(t)} \sum_{i \leq t: a_i = a} Y_i.$$

The ε greedy rule chooses an empirical best arm with probability $1 - \varepsilon$ and a random arm with probability ε . It is easy to implement, but a fixed positive ε continues to select inferior arms at a constant rate, so R_T is generally proportional to T . A decreasing schedule reduces this waste, although choosing the schedule requires care.

The deeper lesson is that useful exploration is not random for its own sake. An arm deserves another observation when its current uncertainty is large enough that it may still be optimal.

Example 13.1: Fixed random exploration

With four arms and $\varepsilon = 0.1$, the rule explores on about $0.1T$ rounds. Even after the best arm is essentially known, roughly three quarters of those exploration rounds choose an inferior arm. The resulting loss remains proportional to T . This simple calculation explains why a constant random exploration rate is unsuitable for a long horizon.

13.3 Upper Confidence Bounds

Suppose rewards are bounded in $[0, 1]$. Hoeffding's inequality motivates the score

$$U_t(a) = \hat{\mu}_a(t-1) + \sqrt{\frac{2 \log t}{N_a(t-1)}}.$$

After each arm has been tried once, UCB selects

$$a_t \in \arg \max_a U_t(a).$$

The empirical mean favors arms that have performed well. The second term favors arms with few observations. It decreases automatically as evidence accumulates.

The score comes from a confidence statement. If arm a has been sampled n times and rewards lie in $[0, 1]$, Hoeffding's inequality gives

$$\Pr \left\{ \mu_a > \hat{\mu}_a + \sqrt{\frac{\log(1/\delta)}{2n}} \right\} \leq \delta.$$

Choosing a small failure probability for every arm and round and applying a union bound produces a time dependent bonus. This derivation explains why the bonus grows slowly with t but shrinks with the number of pulls of that arm. It also makes the assumptions visible: heavy tailed rewards require a different confidence estimate.

Example 13.2: Two candidate actions

Suppose action A has empirical mean 0.70 after 100 observations, while B has mean 0.68 after 4 observations. At $t = 105$, the bonuses are approximately 0.31 for A and 1.53 for B under the displayed formula, so the bound directs attention to B. Constants are often tuned in practice, but the structure is always estimate plus uncertainty.

Under standard stochastic assumptions, UCB satisfies a gap dependent bound of the form

$$\mathbb{E}[R_T] \leq C \sum_{a \neq a^*} \frac{\log T}{\Delta_a} + C',$$

for constants C, C' that depend on the reward assumptions [8]. Logarithmic growth is possible because each inferior arm needs only enough samples to separate its mean from μ^* .

13.4 Thompson Sampling

Thompson sampling represents uncertainty by a posterior distribution. For a Bernoulli arm, start with

$$\mu_a \sim \text{Beta}(\alpha_a, \beta_a).$$

At round t , draw one value $\tilde{\mu}_a$ from each posterior and choose the arm with the largest draw. If the selected arm succeeds, increase α_a by one; if it fails, increase β_a by one. After s_a successes and f_a failures,

$$\mu_a \mid \mathcal{D}_t \sim \text{Beta}(\alpha_a + s_a, \beta_a + f_a).$$

A poorly measured arm has a wide posterior and occasionally produces a high draw, which creates exploration. A well measured strong arm produces high draws consistently, which creates exploitation. Posterior sampling therefore uses probability rather than an explicit bonus to direct exploration.

Example 13.3: A beta posterior update

Start an arm with a $\text{Beta}(1, 1)$ prior. After 7 successes and 3 failures, its posterior is $\text{Beta}(8, 4)$, with mean $8/12$. Another arm with posterior $\text{Beta}(2, 1)$ has less data and greater spread. It can still produce the larger posterior draw even when its posterior mean is lower. That occasional event is the mechanism for exploration.

A Bayesian posterior is not required to be literally true for the method to be useful, but its scale must represent uncertainty reasonably. An overly narrow posterior stops exploration too soon; an overly wide posterior wastes pulls.

13.5 What Carries Forward to Bayesian Optimization

A finite bandit treats unrelated arms separately. Bayesian optimization has a possibly continuous action $x \in \mathcal{X}$ and assumes that observations at nearby or otherwise related points share information. The empirical mean becomes a surrogate mean $m_t(x)$, and the arm specific uncertainty becomes a posterior standard deviation $s_t(x)$. UCB becomes

$$m_t(x) + b_t s_t(x),$$

while Thompson sampling draws an entire plausible function and optimizes it. Thus bandits supply the regret language and the exploration principle; a statistical model supplies generalization across design points.

This bridge also clarifies what changes in the analysis. In a finite bandit, uncertainty is reduced only by pulling the same arm. In BO, one observation can reduce uncertainty at many inputs, so regret depends on the geometry of $\mathcal{X}$ and the complexity of the surrogate. The next two lectures introduce function models that make this sharing explicit.

Lecture 14 Function Approximation and Policy Gradients

14.1 Why Tables Stop Scaling

A tabular method stores one number for every state, or for every state and action pair. When the state space is large or continuous, most entries are never visited. Function approximation replaces the table by a parameter vector w and a map

$$q_w(s, a) \approx Q^*(s, a).$$

Shared parameters allow an observation at one state to change predictions at other states. This is statistical generalization, and it is the same idea that later lets a Gaussian process connect nearby design points.

The main difficulty is that approximation, bootstrapping, and data collected by a changing policy interact. An update can improve one region while worsening another, so convergence is less automatic than in a finite table.

These three ingredients should be kept conceptually separate. Approximation means that many states share parameters. Bootstrapping means that an estimate is updated toward another current estimate. Off policy data mean that the sampling distribution differs from the policy being evaluated. Any one of the three can be manageable, but their combination can produce unstable updates. This is why numerical safeguards are central rather than cosmetic.

14.2 Linear Value Approximation

Choose a feature vector $\phi(s, a) \in \mathbb{R}^d$ and write

$$q_w(s, a) = w^\top \phi(s, a).$$

Given a transition $(S_t, A_t, G_{t+1}, S_{t+1})$, define

$$\delta_t = G_{t+1} + \alpha \max_{a'} q_w(S_{t+1}, a') - q_w(S_t, A_t).$$

Holding the current target fixed during differentiation, a gradient step on the squared prediction error gives

$$w \leftarrow w + \eta_t \delta_t \phi(S_t, A_t).$$

Each component of w is adjusted in proportion to the corresponding feature.

Example 14.1: Features in a gridworld

For action “move right,” take

$$\phi(s, a) = (1, \text{distance to goal}, \text{distance to trap})^\top.$$

If two states have similar distances, their action values use similar feature vectors and therefore share information. The feature choice states which forms of similarity the model can learn.

Linear models are transparent and often stable when features are well chosen. Their limitation is representational: a poor feature map cannot express the value function, regardless of the amount of data.

Feature scaling matters because a gradient step changes w_j in proportion to $\phi_j(S_t, A_t)$. If one feature is measured in thousands and another in fractions, the same learning rate treats them very differently. Centering and scaling continuous features makes the update easier to interpret. A constant feature permits an intercept, and interaction features permit action effects to change with state.

For evaluation of a fixed policy, linear TD seeks a projected Bellman fixed point: the Bellman update may leave the span of the features, so it is projected back into that span. This explains approximation error geometrically. The algorithm can converge perfectly within the feature space and still miss the true value function.

14.3 Nonlinear Q Approximation

A neural network replaces $w^\top \phi(s, a)$ by a nonlinear map $q_w(s, a)$. A deep Q network is trained toward the target

$$G_{t+1} + \alpha \max_{a'} q_{\bar{w}}(S_{t+1}, a'),$$

where $\bar{w}$ is a slowly updated copy of the parameters. Holding this target network fixed for several updates reduces feedback between the current estimate and its own target. A replay buffer stores past transitions and samples them in mixed order, reducing the strong temporal dependence of consecutive data.

These devices do not change the Bellman idea. They are numerical methods for making repeated approximate Bellman updates more stable. The resulting model can be powerful, but its uncertainty is not automatically calibrated, which is important when decisions require directed exploration.

A replay buffer changes the empirical distribution used by the update. If rare but important transitions are almost absent, random replay cannot invent them. Conversely, repeatedly sampling a small set of extreme transitions can bias training toward those cases. The data collection policy and the training sample therefore remain linked even when observations are stored and shuffled.

The target network introduces a second time scale. The current network moves on every gradient step, while the target moves only occasionally or through a slow average. This makes the regression target more nearly fixed during a short block of optimization.

14.4 Differentiable Policies

Instead of estimating action values and then taking a maximum, a policy method models action probabilities directly:

$$\pi_\theta(a \mid s), \quad \sum_a \pi_\theta(a \mid s) = 1.$$

Let

$$J(\theta) = \mathbb{E}_{\pi_\theta} \left[\sum_{t=0}^{\infty} \alpha^t G_{t+1} \right].$$

For any differentiable positive probability,

$$\nabla_\theta \pi_\theta(a \mid s) = \pi_\theta(a \mid s) \nabla_\theta \log \pi_\theta(a \mid s).$$

This log derivative identity moves the gradient from an unknown trajectory probability to the known policy model. The policy gradient theorem then yields an expectation involving $\nabla_\theta \log \pi_\theta(A_t \mid S_t)$ multiplied by a measure of future return.

For a trajectory $\tau = (s_0, a_0, s_1, a_1, \dots)$, the environment transition probabilities do not depend on θ . Therefore

$$\nabla_\theta \log p_\theta(\tau) = \sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t).$$

Multiplying by the trajectory return and taking expectations gives a gradient estimator without differentiating the unknown environment. This is the main reason policy gradients are useful for simulators and real systems whose transition mechanism is not available in analytic form.

For a continuous action, a simple policy is Gaussian with mean $\mu_\theta(s)$ and a positive standard deviation. The mean controls the preferred action and the standard deviation controls random exploration. Both can be learned, subject to any action constraints.

14.5 REINFORCE and Actor and Critic Methods

For an episode, let

$$U_t = \sum_{k=t}^{\tau-1} \alpha^{k-t} G_{k+1}$$

be the return after time t . REINFORCE uses the update

$$\theta \leftarrow \theta + \eta \sum_{t=0}^{\tau-1} \alpha^t \nabla_{\theta} \log \pi_{\theta}(A_t | S_t) [U_t - b(S_t)],$$

where $b(S_t)$ is a baseline. A baseline that does not depend on the sampled action leaves the expected gradient unchanged and can reduce variance.

An actor and critic method learns the policy parameters θ and a value model v_w . The critic forms the temporal difference

$$\delta_t = G_{t+1} + \alpha v_w(S_{t+1}) - v_w(S_t),$$

and the actor updates in the direction

$$\theta \leftarrow \theta + \eta_{\theta} \alpha^t \delta_t \nabla_{\theta} \log \pi_{\theta}(A_t | S_t).$$

The critic estimates whether the outcome was better than expected; the actor makes the chosen action more or less likely. This completes the progression from exact tables to statistical approximations of values and policies.

The baseline in REINFORCE and the critic in an actor and critic method serve related roles. Both compare the observed return with an expected return so that updates respond to advantage rather than raw reward level. A perfect baseline would remove all variation due only to the starting state, but it need not be perfect to help.

Policy gradient methods optimize the parameterized policy, not the set of all possible policies. Their success therefore depends on representation just as value approximation does. The next lecture shifts the same question from representing a policy to representing an unknown response surface for design.

Lecture 15 Experimental Design, Contraction, and Regret

15.1 Prediction and Decision Goals

A design chooses inputs $x_1, \dots, x_n \in \mathcal{X}$ at which data are collected. In a fixed design all inputs are selected in advance. In an adaptive design, x_{t+1} is a function of the current data

$$\mathcal{D}_t = \{(x_i, y_i) : 1 \leq i \leq t\}.$$

The observation model used in the rest of the notes is

$$y_t = f_0(x_t) + \xi_t,$$

where f_0 is the true response and ξ_t is observation noise.

Different design goals can select different points. Estimating f_0 over all of $\mathcal{X}$ favors broad coverage. Estimating a parameter favors directions that identify that parameter. Optimization concentrates on regions that may contain a maximizer. Before choosing a criterion, state which final decision the data must support.

For example, suppose a response is nearly flat over most of $\mathcal{X}$ but has a small region with a possible high value. A space filling design may spend many points estimating the flat region accurately. An optimization design may instead spend most of its budget deciding whether the small region truly contains a better point. Neither design is universally superior; they answer different loss functions.

The design distribution is part of the statistical problem. An estimator can have an excellent rate under a dense fixed grid and fail under a policy that never visits an important region. Statements about contraction must therefore be read together with assumptions on where the data are collected.

15.2 Linear Designs and Information Matrices

Consider

$$y_i = \phi(x_i)^\top \theta_0 + \xi_i, \quad \mathbb{E}(\xi_i \mid x_i) = 0, \quad \text{Var}(\xi_i \mid x_i) = \sigma^2.$$

Let Φ_n have rows $\phi(x_i)^\top$ and define

$$M_n = \Phi_n^\top \Phi_n = \sum_{i=1}^n \phi(x_i) \phi(x_i)^\top.$$

When M_n is invertible, the least squares covariance is $\sigma^2 M_n^{-1}$. A D optimal design makes $\det(M_n)$ large, reducing the volume of a parameter confidence ellipsoid. An A optimal design makes $\text{tr}(M_n^{-1})$ small, reducing average coordinate variance.

Example 15.1: Endpoints identify a line

Let $\phi(x) = (1, x)^\top$ for $x \in [-1, 1]$ and choose two points. Then

$$M_2 = \begin{pmatrix} 2 & x_1 + x_2 \\ x_1 + x_2 & x_1^2 + x_2^2 \end{pmatrix}, \quad \det(M_2) = (x_1 - x_2)^2.$$

The determinant is largest at $x_1 = -1$ and $x_2 = 1$. The endpoints separate the intercept from the slope; two nearly equal points do not.

The eigenvalues of M_n describe information in orthogonal parameter directions. If the smallest eigenvalue is near zero, some linear combination of parameters is weakly identified even when the determinant is not exactly zero. The condition number

$$\kappa(M_n) = \frac{\lambda_{\max}(M_n)}{\lambda_{\min}(M_n)}$$

measures this imbalance. A well conditioned design avoids a direction with very large variance.

Prediction variance at a new input x is

$$\text{Var}\{\phi(x)^\top \hat{\theta} \mid x_1, \dots, x_n\} = \sigma^2 \phi(x)^\top M_n^{-1} \phi(x).$$

Thus parameter criteria and prediction criteria are linked through the same information matrix, but they weight its directions differently.

15.3 Adaptive Design as an Information State

After $\mathcal{D}_t$, choose x_{t+1} and draw

$$Y_{t+1} \sim Q(\cdot \mid \mathcal{D}_t, x_{t+1}).$$

The next state is

$$\mathcal{D}_{t+1} = \mathcal{D}_t \cup \{(x_{t+1}, Y_{t+1})\}.$$

Its conditional distribution depends on the past only through $\mathcal{D}_t$ and the current design point. Therefore an adaptive design is an MDP whose state is information rather than a physical condition.

For terminal utility $U(\mathcal{D}_T)$, define $V_0(\mathcal{D}) = U(\mathcal{D})$ and

$$V_m(\mathcal{D}) = \max_{x \in \mathcal{X}} \mathbb{E}[V_{m-1}(\mathcal{D} \cup \{(x, Y)\}) \mid \mathcal{D}, x].$$

This exact recursion values both the immediate observation and the decisions it will enable. Acquisition functions used in BO are usually simpler one step approximations to this calculation.

To make the terminal utility concrete, let

$$U(\mathcal{D}_T) = - \int_{\mathcal{X}} \{\hat{f}_T(x) - f_0(x)\}^2 d\nu(x)$$

for estimation, or let

$$U(\mathcal{D}_T) = f_0(\hat{x}_T)$$

for optimization. The first values global learning, while the second values a good final recommendation. Although f_0 is unknown, a Bayesian model replaces these utilities by posterior expectations. The resulting optimal design can change dramatically with the choice of terminal loss.

The exact information state need not literally store every data pair. Any summary that determines the predictive law is sufficient. In linear Gaussian models this may be a posterior mean and covariance; in a GP it is represented computationally by the data and kernel factorization.

15.4 Regret Measures Decision Loss

Let

$$x^* \in \arg \max_{x \in \mathcal{X}} f_0(x).$$

For sampled points $x_1, \dots, x_T$ and a final recommendation $\hat{x}_T$, define

$$r_t = f_0(x^*) - f_0(x_t), \quad R_T = \sum_{t=1}^T r_t,$$

and

$$r_T^s = f_0(x^*) - f_0(\hat{x}_T).$$

Cumulative regret charges every poor action during learning. Simple regret charges only the final recommendation. In an online treatment problem, the path matters. In an offline simulator, exploratory evaluations may be harmless and simple regret is often the more relevant target.

Expected regret averages over data and algorithmic randomness. A high probability bound controls regret on an event whose probability is at least $1 - \delta$. Bayesian regret also averages over a prior on f . These are related but distinct statements, so a theorem should say which probability law is being used.

Simple regret can be based on the best sampled point or on a separate recommendation such as $\arg \max_{x \in \mathcal{X}} m_T(x)$. With noisy observations, the largest observed y_t is often an optimistic noise realization, so the posterior mean recommendation is usually easier to justify.

15.5 Uniform Contraction Gives Simple Regret

A contraction statement describes how an estimated function approaches the truth. Suppose $\hat{f}_n$ satisfies

$$\|\hat{f}_n - f_0\|_\infty \leq \epsilon_n$$

on an event whose probability approaches one, and recommend $\hat{x}_n \in \arg \max_{x \in \mathcal{X}} \hat{f}_n(x)$.

Proposition

On this event,

$$f_0(x^*) - f_0(\hat{x}_n) \leq 2\epsilon_n.$$

Proof. Add and subtract the fitted values:

$$\begin{aligned} f_0(x^*) - f_0(\hat{x}_n) &= \{f_0(x^*) - \hat{f}_n(x^*)\} \\ &\quad + \{\hat{f}_n(x^*) - \hat{f}_n(\hat{x}_n)\} \\ &\quad + \{\hat{f}_n(\hat{x}_n) - f_0(\hat{x}_n)\}. \end{aligned}$$

The middle term is nonpositive because $\hat{x}_n$ maximizes $\hat{f}_n$. Each remaining term is at most ϵ_n . $\square$

Thus uniform contraction converts directly to simple regret. An L^2 rate does not give the same conclusion without extra assumptions: a fitted surface can have small average error yet miss a narrow high peak. The statistical norm must control the part of the function that determines the decision [9–11].

A posterior contraction statement is commonly written as

$$\Pi(\|f - f_0\|_\infty > M\epsilon_n \mid \mathcal{D}_n) \longrightarrow 0$$

in probability under data generated from f_0 , for a sufficiently large constant M . This describes the entire posterior, not automatically the posterior mean. A recommendation based on the mean needs a corresponding mean error bound, while a recommendation based on a posterior draw needs a statement that covers that draw.

Function value error can also control optimizer location when the optimum is well separated. Suppose, for all x in a neighborhood of x^* ,

$$f_0(x^*) - f_0(x) \geq c\|x - x^*\|_2^\alpha.$$

Whenever $\hat{x}_n$ lies in this neighborhood, combining this condition with simple regret at most $2\epsilon_n$ gives

$$\|\hat{x}_n - x^*\|_2 \leq \left(\frac{2\epsilon_n}{c}\right)^{1/\alpha}.$$

Without such curvature or margin information, a small value loss need not imply that the estimated optimizer is close in location.

The failure of L^2 control can be visualized by a bump of width w and height one. Missing the bump produces squared integral error of order w , which can be tiny, yet the optimizer value is wrong by one. Uniform error rules out this possibility because it controls the peak itself.

15.6 Sequential Widths Give Cumulative Regret

Suppose that before round t a confidence event gives

$$|f_0(x) - m_{t-1}(x)| \leq b_t s_{t-1}(x) \quad \text{for every } x \in \mathcal{X},$$

and choose x_t to maximize $m_{t-1}(x) + b_t s_{t-1}(x)$. Then

$$r_t \leq 2b_t s_{t-1}(x_t).$$

Indeed, optimism upper bounds $f_0(x^*)$ by the selected upper score, while the lower confidence bound controls $f_0(x_t)$. If b_t is nondecreasing, Cauchy–Schwarz gives

$$R_T \leq 2b_T \sum_{t=1}^T s_{t-1}(x_t) \leq 2b_T \sqrt{T \sum_{t=1}^T s_{t-1}^2(x_t)}.$$

For a Gaussian process, the sum of sampled variances is related to the information gained from the data, leading to bounds of the form $R_T \lesssim b_T \sqrt{T \gamma_T}$, where γ_T measures effective model complexity [12, 13].

For Gaussian noise, information gain for a selected set A can be written

$$I(\mathbf{y}_A; f_A) = \frac{1}{2} \log \det \left(I + \sigma_\xi^{-2} K_A \right),$$

and γ_T is the largest such quantity over sets of T inputs. The log determinant counts how many sufficiently distinct directions of the function have been learned. Repeated highly correlated observations add less information than observations that reveal new directions.

The width argument and posterior contraction are therefore complementary. Contraction asks how the whole posterior concentrates after a design has been used. Regret analysis asks whether the policy chooses points whose widths and losses shrink quickly enough along the path. One can have contraction under a good fixed design without a low regret adaptive policy, or a low simple regret policy without accurate global estimation.

The distinction is essential. Contraction of a final estimator supports simple regret. Cumulative regret also needs valid uncertainty widths along the adaptively chosen path. A proof must check conditional noise assumptions, coverage of competitive regions, and confidence statements that remain valid when each x_t depends on earlier data. In words,

$$\text{function contraction} + \text{decision rule} + \text{adaptive coverage} \implies \text{regret control}.$$

Lecture 16 Gaussian Process Surrogates

16.1 A Prior Over Functions

A Gaussian process places a probability distribution on an unknown function $f : \mathcal{X} \rightarrow \mathbb{R}$ [5]. We write

$$f \sim \text{GP}(m_0, k),$$

where $m_0(x) = \mathbb{E}[f(x)]$ and $k(x, x') = \text{Cov}(f(x), f(x'))$. The definition means that for any finite inputs $x_1, \dots, x_n$,

$$(f(x_1), \dots, f(x_n))^\top \sim N(\mathbf{m}_0, K_n),$$

with $[\mathbf{m}_0]_i = m_0(x_i)$ and $[K_n]_{ij} = k(x_i, x_j)$.

The process is infinite dimensional, but every calculation uses ordinary multivariate normal conditioning. Linear algebra therefore provides the main computational language.

A GP prior does not say that the unknown physical function is random in a literal repeated world sense. It is a probability model for uncertainty before and after observations. The mean m_0 expresses a baseline, while the kernel expresses which deviations from that baseline are plausible. Centering the responses and using $m_0 = 0$ is common, but the centering step should be stated.

For any coefficients $c_1, \dots, c_n$, validity requires

$$\sum_{i,j=1}^n c_i c_j k(x_i, x_j) \geq 0.$$

This is exactly the condition that every kernel matrix can be a covariance matrix. Strict positive definiteness is helpful numerically but is not part of the basic definition.

16.2 Kernels and Shared Information

A valid kernel produces a positive semidefinite matrix for every finite set of inputs. Its diagonal $k(x, x)$ is prior variance; its off diagonal entries state how much learning at one input changes beliefs at another. For example, the squared exponential kernel

$$k(x, x') = \sigma_f^2 \exp\left(-\frac{\|x - x'\|_2^2}{2\ell^2}\right)$$

uses a length scale ℓ . Large ℓ links distant points and favors smooth functions; small ℓ permits faster variation.

Observations follow

$$y_i = f(x_i) + \xi_i, \quad \xi_i \stackrel{\text{ind}}{\sim} N(0, \sigma_\xi^2).$$

Noise adds $\sigma_\xi^2 I_n$ to the covariance of the data. It does not add to the covariance of the latent function itself.

The kernel parameters have direct interpretations. The signal scale σ_f^2 controls vertical variation. The length scale ℓ controls how quickly correlation decays with distance. In several dimensions, separate length scales can reveal that some coordinates matter more than others. The noise variance controls how strongly the posterior follows individual observations.

Inputs should usually be put on comparable numerical scales before fitting length scales. Otherwise a one unit change in one coordinate may mean meters while a one unit change in another means milliseconds, and Euclidean distance has no coherent interpretation. Standardizing responses similarly makes signal and noise scales easier to estimate.

16.3 Posterior Conditioning Formulas

Let

$$\mathbf{y}_t = (y_1, \dots, y_t)^\top, \quad \mathbf{k}_t(x) = (k(x_1, x), \dots, k(x_t, x))^\top,$$

and let $K_t = [k(x_i, x_j)]_{i,j=1}^t$. Also set $\mathbf{m}_{0,t} = (m_0(x_1), \dots, m_0(x_t))^\top$. Conditional Gaussian formulas give

$$m_t(x) = m_0(x) + \mathbf{k}_t(x)^\top (K_t + \sigma_\xi^2 I_t)^{-1} (\mathbf{y}_t - \mathbf{m}_{0,t}),$$

and

$$s_t^2(x) = k(x, x) - \mathbf{k}_t(x)^\top (K_t + \sigma_\xi^2 I_t)^{-1} \mathbf{k}_t(x).$$

In computation, solve a linear system rather than forming the inverse. The posterior mean is a linear combination of observed residuals. The posterior variance equals prior variance minus the amount explained by correlation with the data.

These formulas follow from one block Gaussian vector:

$$\begin{pmatrix} \mathbf{y}_t \\ f(x) \end{pmatrix} \sim N \left(\begin{pmatrix} \mathbf{m}_{0,t} \\ m_0(x) \end{pmatrix}, \begin{pmatrix} K_t + \sigma_\xi^2 I_t & \mathbf{k}_t(x) \\ \mathbf{k}_t(x)^\top & k(x, x) \end{pmatrix} \right).$$

Conditioning the last coordinate on the first block gives the displayed mean and variance. Students familiar with the Schur complement can recognize the variance as the remaining diagonal block after linear prediction from the data.

Dimension checking is a reliable safeguard. The vector $\mathbf{k}_t(x)$ has length t , the matrix solve returns a length t vector, and the final quadratic form is a scalar. A transposed data matrix or a missing noise term is a common source of errors.

16.4 Posterior Uncertainty and Updates

The quantity $s_t^2(x)$ is uncertainty about the latent value $f(x)$. A future noisy observation has variance

$$\text{Var}(Y_{t+1} \mid \mathcal{D}_t, x) = s_t^2(x) + \sigma_\xi^2.$$

This distinction matters in design. Repeated measurements can reduce uncertainty about $f(x)$, but every new measurement still contains observation noise. With zero noise, the posterior variance is zero at an observed input; with positive noise, it generally remains positive.

Posterior uncertainty is also local. It shrinks strongly near observations that are highly correlated with x and weakly in unrelated regions. This is why the kernel and the chosen design jointly determine contraction.

There are also two kinds of uncertainty that a simple GP model does not separate automatically. Posterior variance reflects uncertainty under the chosen kernel and parameter values. If that kernel is badly misspecified, the reported width may be narrow even where the true error is large. Hyperparameter estimation uncertainty can be represented hierarchically or assessed through sensitivity analysis, but a plug in fit should not be mistaken for a guarantee.

Credible intervals are posterior probability statements. Frequentist coverage requires additional assumptions. BO regret proofs therefore choose β_t or another inflation factor so that the band is valid uniformly over relevant inputs and rounds.

Example 16.1: One observation, one prediction

Assume zero prior mean, $k(x, x) = k(x_1, x_1) = 1$, $k(x_1, x) = 0.6$, $\sigma_\xi^2 = 0.25$, and $y_1 = 1$. Then

$$m_1(x) = \frac{0.6}{1 + 0.25}(1) = 0.48,$$

and

$$s_1^2(x) = 1 - \frac{0.6^2}{1.25} = 0.712.$$

The future observation variance is $0.712 + 0.25 = 0.962$. The mean moves toward the observed positive response, while uncertainty falls only by the fraction supported by the correlation 0.6.

At $x = x_1$, the same calculation gives $s_1^2(x_1) = 1 - 1/1.25 = 0.20$: noise prevents complete interpolation.

The update can also be read as linear regression with one predictor. The coefficient $0.6/1.25$ discounts the observed response because the correlation is not perfect and because the observation contains noise. If the noise variance approached zero, the coefficient would become 0.6. If the correlation approached zero, the posterior mean at x would remain at its prior mean.

Example 16.2: Two observations and matrix solves

Continue to assume zero prior mean. Suppose

$$K_2 = \begin{pmatrix} 1 & 0.5 \\ 0.5 & 1 \end{pmatrix}, \quad \sigma_\xi^2 = 0.1, \quad \mathbf{y}_2 = \begin{pmatrix} 1 \\ 0 \end{pmatrix},$$

and $\mathbf{k}_2(x) = (0.8, 0.3)^\top$. To compute $m_2(x)$, solve

$$\begin{pmatrix} 1.1 & 0.5 \\ 0.5 & 1.1 \end{pmatrix} u = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$$

and take $\mathbf{k}_2(x)^\top u$. This is numerically preferable to writing the matrix inverse and makes the role of each dimension explicit.

16.5 Design by Variance Reduction

Before observing Y_{t+1} at a candidate a , the posterior variance after that measurement is already determined:

$$s_{t+1}^2(x) = s_t^2(x) - \frac{c_t(x, a)^2}{s_t^2(a) + \sigma_\xi^2},$$

where $c_t(x, a) = \text{Cov}(f(x), f(a) \mid \mathcal{D}_t)$. The realized response changes the posterior mean, not this variance reduction. A learning oriented design can therefore choose a to reduce

$$\text{IVAR}(\mathcal{D}_t) = \int_{\mathcal{X}} s_t^2(x) d\nu(x),$$

where ν weights regions of practical interest. Sampling where variance is large is not always best: a moderately uncertain point that is correlated with many locations may reduce integrated variance more. BO changes the criterion again by valuing uncertainty mainly where the optimum might lie.

Example 16.3: Integrated variance on three points

Suppose the current variances on three equally weighted points are 0.20, 0.50, 0.80, so IVAR is 0.50. Candidate A would change them to 0.10, 0.30, 0.70, giving IVAR 0.3667. Candidate B would change them to 0.18, 0.45, 0.20, giving IVAR 0.2767. Although B barely helps the first two points, its strong effect on the third gives the larger global reduction.

A Cholesky factorization of $K_t + \sigma_\xi^2 I_t$ gives a stable solve; its cubic cost motivates sparse approximations for large t . Numerical jitter can stabilize nearly duplicate inputs and should be distinguished from observation noise.

Lecture 17 Bayesian Optimization: Information States and Acquisition Rules

17.1 The Bayesian Optimization Loop and Information State

Bayesian optimization seeks a maximizer of an expensive unknown function $f_0 : \mathcal{X} \rightarrow \mathbb{R}$ using a small number of evaluations [7, 14, 15]. At round t ,

$$x_t \in \mathcal{X}, \quad y_t = f_0(x_t) + \xi_t,$$

and the data become $\mathcal{D}_t = \mathcal{D}_{t-1} \cup \{(x_t, y_t)\}$. A standard loop is: fit or update the surrogate, compute an acquisition score, optimize that score, collect the next response, and repeat.

The surrogate mean summarizes promising values. Its uncertainty summarizes what remains plausible. An acquisition function converts those two summaries into the next design decision.

A minimal algorithm is

1. choose an initial design and collect $\mathcal{D}_{t_0}$;
2. fit the GP mean, kernel, and noise model;
3. compute $a_t(x)$ over feasible x ;
4. select x_{t+1} , observe y_{t+1} , and update the posterior;
5. stop at the budget and report a final recommendation.

The acquisition maximizer and the final recommendation need not be the same. The former may be deliberately uncertain; the latter is usually chosen for high estimated value.

BO is most useful when evaluating f_0 is much more costly than optimizing the acquisition. If thousands of true evaluations are cheap, ordinary numerical optimization or dense experimentation may be simpler.

Given $\mathcal{D}_t$ and a proposed x , the posterior predictive distribution of the next response is determined. Hence $\mathcal{D}_t$ is a Markov information state and

$$\mathcal{D}_{t+1} = \mathcal{D}_t \cup \{(x, Y_{t+1})\}$$

is its controlled transition. An exact finite horizon policy would compare expected terminal utility after all remaining measurements. This is usually computationally expensive because the state is a growing data set.

An acquisition function $a_t(x)$ is a tractable score for one proposed point. The rule

$$x_{t+1} \in \arg \max_{x \in \mathcal{X}} a_t(x)$$

is called myopic because it evaluates the next experiment directly rather than solving the full multistep Bellman recursion. Different acquisitions encode different approximations to the value of information.

For example, a one step terminal utility equal to the best posterior value after the next observation leads to a value of information calculation. UCB replaces that expectation by an optimistic bound. EI computes one specific expected terminal gain. Thompson sampling randomizes according to which function is plausible. All use the same information state, but they answer different surrogate decision problems.

A fully nonmyopic policy can be important when only a few measurements remain and one measurement can set up a much more valuable later measurement. Its state and action spaces are usually too large for exact dynamic programming, which is why one step acquisitions dominate practice.

17.2 Upper Confidence Bound

For a GP posterior with mean m_t and standard deviation s_t , the upper confidence rule is

$$a_t^{\text{UCB}}(x) = m_t(x) + \sqrt{\beta_{t+1}} s_t(x), \quad x_{t+1} \in \arg \max_{x \in \mathcal{X}} a_t^{\text{UCB}}(x).$$

The mean term exploits current knowledge; the width term explores plausible high values. To match Lecture 15, set $b_{t+1} = \sqrt{\beta_{t+1}}$. The parameter β_{t+1} controls optimism and may grow with time in theoretical analyses [12].

Example 17.1: One upper confidence decision

At candidates $x^{(1)}$ and $x^{(2)}$, suppose

$$(m_t, s_t)(x^{(1)}) = (1.0, 0.2), \quad (m_t, s_t)(x^{(2)}) = (0.8, 0.6).$$

With $\sqrt{\beta_{t+1}} = 2$, the scores are 1.4 and 2.0, so the second point is selected. Its mean is lower, but its plausible upper value is larger.

On a valid confidence event, the argument from Lecture 15 gives

$$r_{t+1} \leq 2\sqrt{\beta_{t+1}} s_t(x_{t+1}).$$

Thus the same width that drives exploration also controls instantaneous regret.

The scale β_t should not be read as a generic confidence level without checking assumptions. On a finite candidate set it can be chosen by a union bound. On a continuous domain, smoothness and covering arguments or RKHS assumptions are needed to make a simultaneous statement. In applied work, $\sqrt{\beta_t}$ is often treated as an exploration multiplier and examined by sensitivity analysis.

UCB is invariant to adding the same constant to all means, but not to rescaling the response. Standardizing y makes the exploration multiplier more portable. Constraints can be handled by maximizing the score only over the feasible set, provided feasibility is known.

17.3 Expected Improvement

Choose an incumbent level τ_t . In a noiseless problem it is often the best observed value. With noisy data it is safer to use a best posterior mean or another stated estimate of the best latent value. Define improvement by

$$I_t(x) = \{f(x) - \tau_t\}_+.$$

If $f(x) \mid \mathcal{D}_t \sim N(m_t(x), s_t^2(x))$ and $s_t(x) > 0$, let

$$z_t(x) = \frac{m_t(x) - \tau_t}{s_t(x)}.$$

Then

$$a_t^{\text{EI}}(x) = \mathbb{E}[I_t(x) \mid \mathcal{D}_t] = \{m_t(x) - \tau_t\} \Phi(z_t(x)) + s_t(x) \varphi(z_t(x)),$$

where Φ and φ are the standard normal distribution function and density. If $s_t(x) = 0$, EI is $\{m_t(x) - \tau_t\}_+$.

The first term rewards a mean above the incumbent. The second rewards uncertainty near the improvement boundary. EI therefore balances promise and uncertainty without a separate exploration constant, although its behavior depends on how τ_t is defined.

The formula is obtained by writing $f(x) = m_t(x) + s_t(x)Z$ with $Z \sim N(0, 1)$ and integrating only over $Z > -z_t(x)$. Two limiting cases help interpret it. When uncertainty is tiny, EI approaches

$\{m_t(x) - \tau_t\}_+$. When the mean equals the incumbent, EI is $s_t(x)/\sqrt{2\pi}$, so uncertainty alone can create positive value.

With noisy observations, using the largest raw response as τ_t can suppress EI after one unusually high noise realization. Alternatives include the best posterior mean among sampled points, a posterior distribution over the incumbent, or a noisy EI calculation that integrates over latent values. The chosen definition should be stated because it changes the acquisition.

17.4 Probability of Improvement and Posterior Sampling

Probability of improvement uses

$$a_t^{\text{PI}}(x) = \Pr\{f(x) > \tau_t \mid \mathcal{D}_t\} = \Phi(z_t(x)).$$

It asks only whether improvement occurs, not how large it is, so it can focus too narrowly near the incumbent. A margin $\tau_t + \kappa$ can encourage larger improvements, but introduces a tuning parameter.

Posterior sampling draws a function $\tilde{f}_t \sim \mathcal{L}(f \mid \mathcal{D}_t)$ and chooses

$$x_{t+1} \in \arg \max_{x \in \mathcal{X}} \tilde{f}_t(x).$$

This is the continuous design analogue of Thompson sampling. Sampling a whole function preserves posterior dependence across inputs, which is essential: the values at different design points are not independent arms.

Example 17.2: PI, EI, and UCB can disagree

Let the incumbent be 1.0. Candidate A has $(m, s) = (1.05, 0.05)$ and candidate B has $(m, s) = (0.95, 0.30)$. A has the larger probability of a small improvement. B can have the larger expected improvement because its possible gain is much larger, and it can also have the larger UCB for a sufficiently large exploration multiplier. The disagreement reflects different utilities, not an arithmetic error.

Posterior sampling has no deterministic acquisition surface until a function is drawn. Repeating the draw would produce different decisions. This randomness is purposeful: the frequency with which a region is selected reflects posterior belief that its maximizer lies there.

17.5 Optimizing the Acquisition and Stopping

The acquisition is cheap relative to evaluating f_0 , but it may be nonconvex. On a finite candidate set, evaluate every score. On a continuous domain, use several initial points for a local optimizer, or optimize over a dense candidate set and refine the best candidates. Constraints on feasible experiments must be imposed in this inner optimization.

Useful stopping rules include a fixed evaluation budget, a small maximum posterior standard deviation in the competitive region, a small maximum EI, or stability of the recommended point. None is universal. The stopping rule should match the final loss: surface estimation, simple regret, cumulative regret, or the cost of one more experiment.

Gradients of UCB and EI can be obtained when the kernel is differentiable, but a local optimum of the acquisition need not be global. Several starting points or a candidate grid are therefore part of the method rather than an afterthought. For categorical or mixed inputs, direct enumeration, local search over discrete neighbors, or specialized optimizers may be more appropriate than calculus.

A compact comparison is:

rule	main quantity	main choice
UCB	$m_t + \sqrt{\beta_t} s_t$	exploration scale
EI	$\mathbb{E}[(f - \tau_t)_+ \mid \mathcal{D}_t]$	incumbent τ_t
PI	$\Pr(f > \tau_t \mid \mathcal{D}_t)$	improvement margin
posterior sampling	$\tilde{f}_t \mid \mathcal{D}_t$	posterior model

No row is uniformly best; model calibration and the decision objective matter at least as much as the acquisition name.

Lecture 18 A Common View of MDPs and Bayesian Surrogates

18.1 A Common Decision Cycle and Two Kinds of State

All three frameworks repeat the same cycle: observe a state, choose an action, observe an outcome, update, and continue. The objects differ:

model	state before action	action	new observation
MDP	S_t	A_t	(G_{t+1}, S_{t+1})
bandit	$\mathcal{D}_{t-1}$	a_t	Y_t
BO	$\mathcal{D}_{t-1}$	x_t	y_t

A classical bandit has one physical state, but its data summary changes. BO adds a model that shares information across a structured action space.

The common cycle can also be written as a controlled probability law. Given a state I_t and action a_t , an observation Z_{t+1} is drawn from $Q(\cdot \mid I_t, a_t)$ and the update is $I_{t+1} = U(I_t, a_t, Z_{t+1})$. For an MDP, $I_t = S_t$ when the model is known. For a bandit or BO problem, I_t is a data summary. This abstract notation shows that the distinction is not whether a state exists, but what the state must contain.

A physical state describes the system itself: queue length, inventory, patient condition, or score difference. An information state describes what the decision maker knows: counts, parameter estimates, or a posterior distribution. When an MDP model is known, S_t may be enough. When it must be learned, a complete state can include both pieces,

$$(S_t, \mathcal{D}_t).$$

This augmentation restores the Markov property for learning problems. It also clarifies why BO is a control problem even when no physical system evolves: the posterior evolves after every experiment.

A sufficient information state need not be unique. In a Bernoulli bandit with beta priors, posterior success and failure counts are enough; the order of the observations can be discarded. In a linear Gaussian model, a posterior mean and covariance are enough. In a general nonparametric model, retaining all data may be the simplest exact representation. Choosing a compact state is a form of statistical modeling.

When the system has both physical evolution and learning, dropping either part can be costly. A treatment policy may need the patient's current condition and uncertainty about treatment effects. A queue controller may need current queue length and uncertainty about arrival rates. This is the natural meeting point of reinforcement learning and Bayesian experimental design.

18.2 Bellman Values and Acquisition Scores

A Bellman value accounts for all remaining decisions. Let I be the current information state, let a be an action, let Z be the next observation, and let $U(I, a, Z)$ be the state update. If $u(I, a, Z)$ is immediate utility, then

$$V_m(I) = \max_{a \in A(I)} \mathbb{E}[u(I, a, Z) + V_{m-1}\{U(I, a, Z)\} \mid I, a].$$

A bandit index or BO acquisition usually replaces this full look ahead with a one step score. UCB uses an optimistic value, EI uses expected gain over an incumbent, and posterior sampling acts optimally for one sampled model. These rules are computationally simpler, but they encode different approximations to future information.

The difference between exact and one step planning is easiest to see with two measurements remaining. A point with little direct value may reveal which of two regions should receive the final

measurement. A purely myopic rule can miss this option value. Exact dynamic programming accounts for it through the continuation term; a one step acquisition either ignores it or approximates it indirectly.

Conversely, a full Bellman formulation is only useful if its state, predictive law, and terminal utility are credible. A complicated dynamic program built on a poor model can be less reliable than a transparent one step rule.

18.3 Exploration, Uncertainty, and Regret

Exploration has a different source in each model. In an unknown MDP, an action reveals rewards and transitions and also moves the physical state. In a bandit, an action reveals one arm mean. In BO, a design point changes beliefs about a whole region through the kernel or surrogate model.

Example 18.1: The same two choices under two objectives

Suppose two actions have estimated values 0.70 and 0.68, with standard errors 0.02 and 0.10. A current reward rule chooses the first. An optimistic score with multiplier 2 gives 0.74 and 0.88, so it explores the second. If only the final recommendation matters, that experiment may be worthwhile; if each action treats a patient, its immediate regret also matters.

Regret supplies a common benchmark, but its definition follows the model. MDP policy loss compares value functions, bandit regret compares arm means, and BO regret compares function values. Simple regret is terminal; cumulative regret adds losses along the learning path.

The horizon also changes interpretation. In an MDP, a horizon may be time in the physical system. In a bandit or BO problem, it is often an information budget. The same symbol T can therefore count treatments, experiments, or simulator runs. Stating what one round costs prevents confusion about which regret notion is operationally meaningful.

Uncertainty is model specific, but the decision pattern is shared:

$$\text{estimate of value} + \text{allowance for uncertainty}.$$

UCB makes the allowance explicit. Thompson sampling randomizes it. Bellman planning prices uncertainty through future value. Posterior contraction asks whether that allowance can eventually shrink.

18.4 Choosing the Appropriate Model

Use an MDP when actions materially change a physical state whose future matters. Use a bandit when actions can be treated as a finite set of repeated choices and cross action structure is weak or intentionally ignored. Use BO when the action is a structured design variable, evaluations are expensive, and a surrogate can share information across inputs.

Three questions usually resolve the choice:

1. Does today's action change tomorrow's physical state?
2. Is the unknown object a finite list of means or a response function?
3. Are learning actions themselves costly, or does only the final deployed decision matter?

The answers determine the state, the statistical model, and whether cumulative or simple regret is the natural criterion.

Hybrid models are common. A contextual bandit observes covariates before each choice but assumes the action does not affect the next context. A model based RL method may use a GP for an unknown transition or reward function. BO may tune parameters of a policy that will later control an MDP. The common framework helps organize these hybrids without forcing every problem into one label.

A final modeling check is counterfactual: after choosing an action, which future quantities would have been different under another action? If only the current reward changes, a bandit view may suffice. If the physical state changes, use an MDP. If the main change is what is learned about a response surface, use an information state design view.

Lecture 19 Treatment and Inventory as Decision Models

19.1 Begin With the Decision Question

An application does not determine a mathematical model by itself. The same domain can produce an MDP, a bandit, or a BO problem depending on what repeats, what evolves, and what is unknown. The first step is to state the unit of decision, the state observed before that decision, and the benchmark used to judge performance.

A second step is to identify what resets. If every experimental unit begins from the same condition, repeated choices may be treated as independent arms or design points. If a choice carries inventory, health, or capacity into the next period, the physical state cannot be discarded. A third step is to decide whether structure across actions is scientifically credible. A continuous dose model assumes that nearby doses have related effects; a finite bandit does not.

19.2 Treatment Over Time as an MDP

Example 19.1: Sequential treatment

Let S_t summarize a patient's current condition and let A_t be a treatment. The next condition follows $p(s' | s, a)$, and $g(s, a)$ combines health benefit and treatment burden. A policy maps each condition to a treatment and seeks to maximize

$$\mathbb{E} \left[\sum_{t=0}^{T-1} g(S_t, A_t) + g_{\text{term}}(S_T) \right].$$

Because treatment changes future condition, a myopic comparison of immediate responses can be wrong. The physical state and continuation value are central.

For a finite horizon, the Bellman recursion is

$$V_m(s) = \max_{a \in A(s)} \left\{ g(s, a) + \sum_{s'} p(s' | s, a) V_{m-1}(s') \right\}.$$

A treatment with a small current benefit may be optimal if it moves the patient toward a better future condition. Conversely, a treatment with a large short term response may be poor if it increases later risk. This is the reason a sequence of treatment decisions cannot generally be reduced to independent arm means.

If transition and reward models are unknown, the information state must be added to the patient state. This creates an RL problem, and exploration has an ethical cost because the learning actions are real treatments.

If each patient contributes one independent treatment decision and there is no relevant carryover, a finite set of treatments can be modeled as arms with means μ_a . Cumulative regret compares the outcomes of assigned treatments with the best fixed arm.

For a continuous dose $x \in [x_{\min}, x_{\max}]$, write

$$y_t = f_0(x_t) + \xi_t,$$

where f_0 is expected clinical utility. BO can share information across nearby doses and recommend $\hat{x}_T \in \arg \max_{x \in \mathcal{X}} m_T(x)$. This formulation requires explicit safety constraints or a restricted feasible set; an acquisition score alone does not make unsafe exploration acceptable.

Example 19.2: Three treatment formulations

Suppose treatments A, B, and C are given once to independent patients. A bandit estimates three means. If treatment intensity is a continuous dose, BO models a curve $f_0(x)$. If the same patient is treated repeatedly and current dose changes future health, an MDP uses patient state and transition dynamics. The clinical topic is the same, but the data and counterfactual assumptions are not.

Patient covariates can be included in the state or in a response model. If they are observed before a one time treatment and do not evolve in response to the action, the resulting problem lies between a simple bandit and an MDP. The key question remains whether today's action affects tomorrow's decision context.

19.3 Inventory and Pricing as Sequential or Design Problems

Inventory is naturally dynamic. With stock S_t , order A_t , and demand D_t ,

$$S_{t+1} = S_t + A_t - D_t.$$

Holding, shortage, and order costs create the one period cost $c(S_t, A_t)$, and the Bellman equation compares today's order with the value of tomorrow's stock. This is an MDP because the order changes both immediate cost and the next physical state.

A bandit model for order quantities would discard this carryover and is only reasonable when each experiment is reset to the same starting condition. That modeling distinction is more important than the choice of algorithm.

Example 19.3: A two period inventory comparison

Suppose one unit is on hand, demand in each period is either zero or one, and an order arrives immediately. Ordering now changes both the current shortage risk and the stock available next period. The value of an order is therefore not a fixed arm mean. It depends on current inventory and the number of periods remaining.

If demand parameters are unknown, the state can be enlarged to include a posterior or sufficient statistics. Learning demand and controlling stock then occur together. A certainty equivalent method plugs in one demand estimate; a Bayesian dynamic program integrates over the remaining uncertainty.

Suppose each selling period starts with a fresh market and a price a is chosen from a small menu. If the resulting profit has mean μ_a , the problem is a bandit. If price is continuous and expected profit is a smooth response $f_0(x)$, BO is more efficient because observations at similar prices are related.

When inventory is limited, however, price affects future stock. The state must then include remaining inventory, turning the problem into revenue management or an MDP. A useful hybrid uses BO to learn an unknown demand curve while a dynamic program controls inventory under the current estimate.

For a continuous price x , expected profit may be

$$f_0(x) = (x - c) d_0(x),$$

where $d_0(x)$ is mean demand and c is unit cost. A GP can model profit directly, or it can model demand and use the economic formula. The second approach imposes more structure and can support inventory calculations, while the first is simpler if only the best static price matters.

When stock is scarce, the opportunity cost of selling one unit depends on the remaining horizon. The relevant reward is then current margin plus continuation value, not static expected profit. This is another example of a response surface becoming only one component of an MDP.

19.4 Assumptions That Change the Answer

The following comparison makes the modeling choices explicit:

question	state	unknown object	main loss
sequential treatment	S_t	p, g	policy value
independent treatment	$\mathcal{D}_t$	$\{\mu_a\}$	R_T
continuous dose	$\mathcal{D}_t$	$f_0(x)$	r_T^s or R_T
inventory control	S_t	possibly p, g	total cost
continuous price	$\mathcal{D}_t$	$f_0(x)$	r_T^s

The table is a diagnostic, not a taxonomy. Real systems may need both physical and information states, and the correct model should preserve every feature that changes the value of future decisions.

A practical way to test a proposed simplification is to compare two histories that produce the same proposed state. If they imply different predictive laws or different optimal actions, the state is not sufficient. For treatment, the missing item might be prior dose history. For inventory, it might be an outstanding order. For BO, it might be observation noise or replication count. State design is therefore part of model validation.

Lecture 20 Simulation Policy Tuning and Regret

20.1 Inner and Outer Decisions with Policy Parameters

Simulation tuning often contains two levels. The inner model is the stochastic system under a fixed policy. It may be a queue, inventory system, or treatment process and can itself be an MDP. The outer problem chooses which policy parameter to test next. If x indexes a policy, define

$$f_0(x) = \text{expected performance of policy } x.$$

A simulation run produces a noisy evaluation of this function. BO acts on the outer problem; it does not replace the inner state model.

The two levels also have different sources of randomness. Within one run, the inner simulator generates arrivals, failures, demands, or outcomes. Across outer rounds, the tuning rule chooses a new policy parameter in response to previous simulated averages. Treating these levels separately prevents the queue state, the simulation seed, and the BO data set from being mixed into one ambiguous state variable.

The objective may be a finite horizon reward, a steady average, a risk measure, or a constrained performance metric. Whatever is simulated should be written as one scalar response for the outer optimization, with constraints recorded separately if violating them is unacceptable.

Example 20.1: A queue threshold

A queue activates an extra server when its length exceeds threshold $x \in \{1, \dots, 10\}$. For each fixed x , queue length evolves as a Markov chain and the simulation estimates long run net reward $f_0(x)$. The outer data are

$$\mathcal{D}_t = \{(x_i, y_i) : 1 \leq i \leq t\},$$

and the next threshold is selected by an acquisition rule. The inner state is queue length; the outer state is the accumulated simulation evidence.

The parameter may be continuous, for example a reorder level, service target, or coefficient in a control rule. A low dimensional parameterization makes the outer optimization feasible, but it also limits the policies that can be found.

A parameterized policy can be written π_x . The outer function is then

$$f_0(x) = J(\pi_x),$$

where J is the performance criterion of the inner stochastic system. This notation makes the approximation explicit: BO searches the family $\{\pi_x : x \in \mathcal{X}\}$, not all admissible policies. A poor family cannot be repaired by more accurate optimization.

The design space should encode implementable policies. Integer staffing levels, ordered thresholds, and budget constraints can be built into $\mathcal{X}$. Optimizing over infeasible parameters and projecting afterward can change the objective and invalidate the surrogate interpretation.

20.2 Noisy Evaluation and Replication

Let $L(x, \omega)$ be the performance from one simulated random scenario ω , and let

$$f_0(x) = \mathbb{E}[L(x, \omega)].$$

With M_t independent replications at x_t ,

$$y_t = \frac{1}{M_t} \sum_{j=1}^{M_t} L(x_t, \omega_{tj}), \quad \text{Var}(y_t | x_t) = \frac{\sigma^2(x_t)}{M_t}.$$

Replication reduces noise but uses budget that could explore another input. When variance changes with x , the GP observation noise should reflect that change rather than assume a common value.

Using the same random scenarios at two inputs can make their difference more precise. This device is called common random numbers. It creates correlation between evaluations, so that correlation must be represented or handled in the analysis rather than treated as independent noise.

Example 20.2: How replication changes precision

Suppose one run at x has standard deviation 12. With $M = 9$ independent replications, the standard deviation of the sample mean is $12/\sqrt{9} = 4$. Increasing to $M = 36$ lowers it to 2, but uses four times as many runs. The last reduction may be less valuable than testing a new parameter where the posterior uncertainty is much larger.

For moderate replication counts, the central limit theorem often motivates

$$y_t \mid x_t \approx N\left(f_0(x_t), \frac{\sigma^2(x_t)}{M_t}\right).$$

The approximation should be checked when outputs are highly skewed, heavy tailed, or generated from a long run average with strong serial dependence. Batch means or regenerative methods may be needed to estimate the variance of a single long simulation.

20.3 A Bayesian Optimization Workflow

A practical workflow is:

1. choose a small initial design that covers the feasible parameter range;
2. run enough replications to estimate both mean and simulation variance;
3. fit a GP to the mean responses with their observation variances;
4. optimize an acquisition such as UCB or EI;
5. decide whether to explore a new input or replicate a promising noisy one;
6. stop at the budget and recommend a point using the final posterior mean.

The recommendation is commonly

$$\hat{x}_T \in \arg \max_{x \in \mathcal{X}} m_T(x),$$

possibly restricted to inputs that satisfy operational constraints. The initial design and final validation should use random seeds not chosen in response to favorable outcomes.

The choice between a new input and another replication is itself a design decision. A new input improves spatial coverage; a replicate reduces noise at an existing input. When the noise variance is known, the GP update quantifies both effects. When it is estimated, a small number of replications at several initial points is often more informative than many replications at only one point.

The budget should count the true computational cost. If run length varies with x , one evaluation is not one unit of effort. An acquisition can be divided by estimated run time or formulated as expected value per unit cost. This prevents the optimizer from favoring a point only because its expensive simulation has been miscounted.

20.4 Regret, Validation, and the Contraction Connection

If simulations are offline and only the final policy is deployed, simple regret

$$r_T^s = f_0(x^*) - f_0(\hat{x}_T)$$

is the natural optimization loss. Poor intermediate simulations consume computer time but do not directly harm the operating system. Their cost can be represented separately by an evaluation budget.

If each tested policy is deployed while learning, cumulative regret

$$R_T = \sum_{t=1}^T \{f_0(x^*) - f_0(x_t)\}$$

matters as well. This distinction determines whether broad early exploration is acceptable. The same algorithm can look strong under simple regret and weak under cumulative regret.

There is also a distinction between simulation regret and deployment regret. Testing a poor policy only in simulation spends computation. Deploying a poor policy affects customers or operations. A workflow may therefore allow broad simulation exploration but require a conservative validation gate before real use. Combining both stages into one cumulative regret number can hide this important operational boundary.

For offline tuning, an alternative loss is the opportunity cost of the final choice plus the monetary cost of simulation. This makes the stopping decision explicit: continue only while the expected reduction in decision loss exceeds the cost of more runs.

Selection makes the apparent best simulated policy optimistic. After tuning, run fresh independent replications at $\hat{x}_T$ and at useful benchmarks. Report uncertainty for their difference, not only the largest fitted mean. This separates optimization data from evidence used to support deployment.

The final connection follows Lecture 15. If the surrogate contracts uniformly, then maximizing its mean gives a simple regret bound. If policies are deployed sequentially, one also needs calibrated widths along the chosen path to control cumulative regret. Across MDPs, bandits, and BO, the recurring logic is

state representation $\longrightarrow$ uncertainty update $\longrightarrow$ decision rule $\longrightarrow$ decision loss.

A coherent analysis must keep all four links explicit.

The overall course can now be read as one progression. Markov models describe how actions change physical systems. Reinforcement learning estimates values when those models are unknown. Bandits isolate directed exploration. Gaussian processes generalize information across a design space. Bayesian optimization turns that uncertainty into a sequence of expensive experiments. Contraction and regret then provide complementary checks on whether learning has produced a good decision.

References

- [1] Martin L. Puterman. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. John Wiley & Sons, 1994.
- [2] Dimitri P. Bertsekas. *Dynamic Programming and Optimal Control*, volume 2. Athena Scientific, 4 edition, 2012.
- [3] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2 edition, 2018.

- [4] Thomas J. Santner, Brian J. Williams, and William I. Notz. *The Design and Analysis of Computer Experiments*. Springer, 2003.
- [5] Carl Edward Rasmussen and Christopher K. I. Williams. *Gaussian Processes for Machine Learning*. MIT Press, 2006.
- [6] Peter I. Frazier. A tutorial on bayesian optimization. *arXiv preprint arXiv:1807.02811*, 2018.
- [7] Bobak Shahriari, Kevin Swersky, Ziyu Wang, Ryan P. Adams, and Nando de Freitas. Taking the human out of the loop: A review of bayesian optimization. *Proceedings of the IEEE*, 104(1):148–175, 2016.
- [8] Peter Auer, Nicolo Cesa-Bianchi, and Paul Fischer. Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, 47(2–3):235–256, 2002.
- [9] Aad W. van der Vaart and J. Harry van Zanten. Rates of contraction of posterior distributions based on gaussian process priors. *The Annals of Statistics*, 36(3):1435–1463, 2008.
- [10] Subhashis Ghosal and Aad W. van der Vaart. *Fundamentals of Nonparametric Bayesian Inference*. Cambridge University Press, 2017.
- [11] Adam D. Bull. Convergence rates of efficient global optimization algorithms. *Journal of Machine Learning Research*, 12:2879–2904, 2011.
- [12] Niranjana Srinivas, Andreas Krause, Sham M. Kakade, and Matthias W. Seeger. Gaussian process optimization in the bandit setting: No regret and experimental design. In *Proceedings of the 27th International Conference on Machine Learning*, pages 1015–1022, 2010.
- [13] Sattar Vakili, Kia Khezeli, and Victor Picheny. On information gain and regret bounds in gaussian process bandits. In *Proceedings of the 24th International Conference on Artificial Intelligence and Statistics*, volume 130 of *Proceedings of Machine Learning Research*, pages 82–90, 2021.
- [14] Jonas Mockus, Vytautas Tiesis, and Antanas Zilinskas. The application of bayesian methods for seeking the extremum. In *Towards Global Optimization*, volume 2, pages 117–129. North-Holland, 1978.
- [15] Donald R. Jones, Matthias Schonlau, and William J. Welch. Efficient global optimization of expensive black-box functions. *Journal of Global Optimization*, 13(4):455–492, 1998.